

# Bayes-Kalman Filters, Physical World Models, Dynamic Programming and Hidden Markov Models

Directed graphical models over time  
Notes for Probabilistic Models of the Visual Cortex

Alan Yuille

Johns Hopkins University

29 September, 2026

# What this lecture covers

The Bayes-Kalman filter

Modeling the physical world

Dynamic programming

Hidden Markov models

Summary

# What this lecture covers

The colours carry meaning, and they are the same in every equation and every figure drawn for these notes.

- ▶ **Red** marks the hidden state: the position  $x_t$  of the Bayes-Kalman filter, the world state, and the state  $q_t$  of a hidden Markov model.
- ▶ **Blue** marks what is observed: the measurement  $y_t$ , the image, and the observation  $O_t$ .
- ▶ **Orange** marks the prediction, which carries the estimate forward in time before the next observation arrives.
- ▶ **Teal** marks the estimate after the correction, and the best solutions of dynamic programming: the shortest path and the Viterbi path.
- ▶ **Purple** marks the actions.
- ▶ **Olive** marks the choice that looks best at each time separately, and turns out to be inconsistent.

In the graphs drawn for these notes a hollow node is hidden and a filled node is observed. Figures reused from other authors keep their own colours, and their captions say what the colours mean.

## From hidden variables to hidden states that change in time

The lecture on directed graphs wrote a probability distribution as a graph. Each node carries the conditional distribution of its variable given its parents, the nodes with arrows into it, and the joint distribution of all the variables is the product of these conditionals,

$$P(x_1, \dots, x_N) = \prod_{i=1}^N P(x_i \mid \text{Pa}(x_i)),$$

where  $\text{Pa}(x_i)$  is the set of parents of  $x_i$ . In the cue-combination example a single cause  $S$ , such as the depth of a surface, generates two cues  $C_1$  and  $C_2$ , and the graph with arrows  $S \rightarrow C_1$  and  $S \rightarrow C_2$  gives

$$P(S, C_1, C_2) = P(C_1 \mid S) P(C_2 \mid S) P(S).$$

One distribution per node is the whole description of the model.

## From hidden variables to hidden states that change in time

The last lecture learnt the parameters of such models when some variables are hidden: never observed, and inferred at the same time as the parameters are estimated. Expectation-maximisation (EM) alternates the two tasks. The E-step infers the hidden variables with the parameters held fixed, and the M-step re-estimates the parameters with the inferred hidden variables held fixed. The hidden variables were the class labels of k-means, the line processes of the weak membrane, and the hidden neurons of the Boltzmann machine.

In each of those models the hidden variables describe one fixed situation: one set of datapoints, one image, one pattern of activity. The hidden variables of this lecture change in time.

## From hidden variables to hidden states that change in time

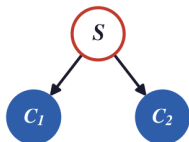
In a video the hidden variable is the state of the world, and it changes from frame to frame: an object moves, a tower of blocks falls. Over time the directed graph becomes a chain. The state  $x_t$  at time  $t$  depends on the state  $x_{t-1}$  one frame earlier, and the image, or any measurement  $y_t$  made from it, depends on the state at the same time.

The new question is: how do we combine observations over time, when the state of the world changes from frame to frame?

Four tools answer it. The Bayes-Kalman filter estimates the current state from all the observations so far, one frame at a time. Models of the physical world use the same graph, with physics as the rule that moves the state forward and graphics as the rule that turns a state into an image. Dynamic programming finds the best whole sequence of states on a chain. Hidden Markov models are the same graph with discrete states, and dynamic programming evaluates them, decodes them and, inside EM, learns them.

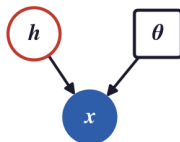
# From hidden variables to hidden states that change in time

**directed graph**



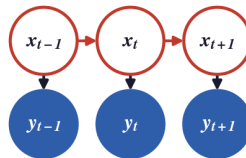
one distribution  
per node

**hidden variables**



learnt by EM

**states in time**



state red,  
observation blue

**Figure 1:** The two earlier ideas this lecture builds on, and the new one. Hollow red nodes are hidden, filled blue nodes observed. A directed graph has one conditional distribution per node: here the hidden cause  $S$  and its two observed cues  $C_1$  and  $C_2$ . With hidden variables, the hidden node  $h$  and the parameters  $\theta$  (the square) generate the data  $x$ , and EM learns  $\theta$ . In this lecture the hidden state  $x_t$  changes in time: each state depends on the one before, and each observation  $y_t$  depends on the state at its own time.

# The perception of motion depends on its history

Visual motion is often studied as the change of the image from one frame to the next. Human perception of motion is also strongly influenced by its history. Anstis and Ramachandran (1987) studied apparent motion, the impression of movement produced by a sequence of stationary flashes, as in a film, and found a visual inertia: an ambiguous display that follows motion in one direction tends to be seen moving on in the same direction. Phenomena of this kind seem to need a temporal coherence prior, a prior that motion changes smoothly over time, in addition to the priors, used in models of motion between a single pair of frames, that favour slow and spatially smooth velocities.

## The perception of motion depends on its history

Watamaniuk, McKee and Grzywacz (1995) showed that humans can detect a single dot moving along a straight trajectory among many dots that move in random directions. In any single frame the target is one dot among many and looks like all the others; what makes it visible is that its direction stays the same over many frames.

The figure illustrates a stimulus of this kind. Thirty-eight noise dots take steps of fixed length in random directions, each step independent of the last, and are reflected at the border of the panel. One target dot moves in a straight line, with steps of the same length, so only its constant direction sets it apart. The first panel is a single frame, in which all thirty-nine dots look alike. The second panel shows six frames: the grey paths of the noise dots wander, the path of the target, in red, is straight, and the black dots are the positions in the last frame.

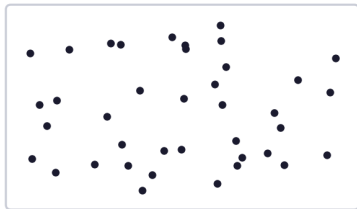
## The perception of motion depends on its history

These classes of phenomena can be addressed by models that make prior assumptions about how motion changes over time. Yuille, Burgi and Grzywacz (1998) built such a model by adapting the Bayes-Kalman filter (Kalman, 1960; Ho and Lee, 1964), which gives an optimal way to combine information over time: a prior on how velocity changes from frame to frame carries the estimate of the motion forward in time, and each new frame corrects it.

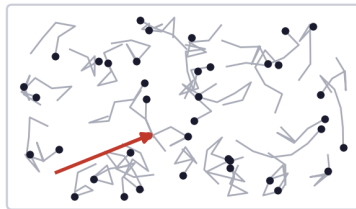
In a model of this kind a dot that moves consistently accumulates evidence frame after frame, because its motion in each new image agrees with what the preceding images predicted. A dot that changes direction at random gains nothing from its history, since each frame contradicts the prediction of the one before.

# The perception of motion depends on its history

**one frame:  
every dot looks alike**



**six frames: the red  
trajectory is straight**



**Figure 2:** A stimulus of the kind used by Watamaniuk, McKee and Grzywacz (1995). Every dot moves the same distance per frame. In one frame the target is one dot among thirty-nine and cannot be told apart. Over six frames the grey paths of the thirty-eight noise dots wander, and the red path of the target is straight; the arrow gives its direction and the black dots are the last frame.

# The Bayes-Kalman filter estimates a state from observations

The task of the Bayes-Kalman filter is to estimate the state  $x_t$  of a system at time  $t$  from a set of observations  $y_t, \dots, y_1$ . The state is the quantity we want to know and cannot see directly; the observation is what a sensor delivers. For example  $x_t$  could be the position of an airplane at time  $t$  and  $y_t$  a noisy measurement of that position, or  $x_t$  the position of an object in a video and  $y_t$  its position as detected in frame  $t$ . Write  $Y_t = \{y_t, \dots, y_1\}$  for all the observations up to time  $t$ . The model has two parts,

$$\text{motion: } P(x_{t+1} | x_t), \quad \text{observation: } P(y_t | x_t).$$

The motion model says how the state changes from one time to the next, and the observation model, a likelihood function, says how the measurement depends on the state. The goal is the distribution  $P(x_t | Y_t)$  of the current state given everything observed so far.

# The Bayes-Kalman filter estimates a state from observations

The states form a Markov chain over time, and with the observations the model is a directed graphical model. Each state has one parent, the state before it, and each observation has one parent, the state at the same time. The product of one conditional per node is the joint distribution of  $T$  states and  $T$  observations,

$$P(x_1, \dots, x_T, y_1, \dots, y_T) = P(x_1) \prod_{t=1}^{T-1} P(x_{t+1} | x_t) \prod_{t=1}^T P(y_t | x_t),$$

where the prior  $P(x_1)$  is the distribution of the first state before any measurement. Two conditional independences follow from the graph and are used throughout this lecture:

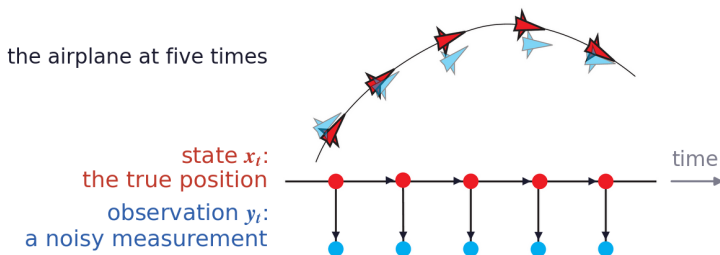
- ▶ given the present state  $x_t$ , the next state  $x_{t+1}$  is independent of the observations  $Y_t$ ;
- ▶ given the state  $x_{t+1}$ , the observation  $y_{t+1}$  is independent of the earlier observations  $Y_t$ .

## The Bayes-Kalman filter estimates a state from observations

Kalman (1960) derived the filter for linear systems, whose state changes by a linear equation plus Gaussian noise, as a recursion that updates a mean and a variance. Ho and Lee (1964) gave the Bayesian formulation, which propagates a whole probability distribution and holds for any motion and observation models; Kalman's filter is its Gaussian case. The general version is called the Bayes-Kalman filter.

In the figure, taken from the course notes (Yuille and Kersten, 2025), the airplane is shown at five times. The red airplanes and red dots are its true positions, the states; the blue airplanes and blue dots are the observations, which are displaced from the true positions by the measurement noise. The arrowheads have been added to the original figure: each state points to the next state and to its own observation, as in the graph.

# The Bayes-Kalman filter estimates a state from observations



**Figure 3:** Tracking an airplane (Yuille and Kersten, 2025, Figure 4, first part, with arrowheads added). The red airplanes and dots are the true positions at five times, the blue ones the observations. Each red state points to the next state and to its own blue observation.

## Prediction and correction update the state's distribution

The purpose of Bayes-Kalman is to compute  $P(x_t | Y_t)$  at every time  $t$ . It does so by repeating two steps, called prediction and correction. The prediction uses the motion model to carry the distribution forward one step, before the next observation is seen,

$$\text{prediction: } P(x_{t+1} | Y_t) = \int dx_t P(x_{t+1} | x_t) P(x_t | Y_t).$$

Every possible current state  $x_t$  is moved by the motion model and weighted by how probable it is given the observations so far; the integral adds up all the ways of arriving at  $x_{t+1}$ .

The correction integrates the new observation  $y_{t+1}$  by Bayes' rule, with the prediction as the prior,

$$\text{correction: } P(x_{t+1} | Y_{t+1}) = \frac{P(y_{t+1} | x_{t+1}) P(x_{t+1} | Y_t)}{P(y_{t+1} | Y_t)},$$

where the denominator  $P(y_{t+1} | Y_t) = \int dx_{t+1} P(y_{t+1} | x_{t+1}) P(x_{t+1} | Y_t)$  makes the result a distribution.

## Prediction and correction update the state's distribution

The filter is initialised with the prior  $P(x_1)$  on the position at the start of the sequence and the first observation,

$$P(x_1 | y_1) = \frac{P(y_1 | x_1) P(x_1)}{P(y_1)},$$

and then the prediction and the correction are run at every frame.

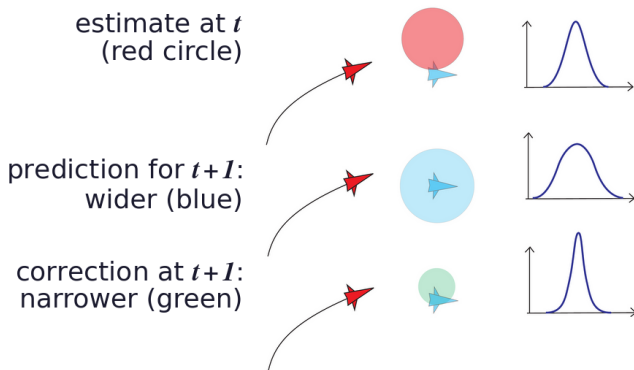
The effect of prediction is to introduce uncertainty about the state: the motion model is itself uncertain, and for the motion models of this lecture, which add noise to the state, the predicted distribution is wider than the estimate it started from. The correction reduces the uncertainty by providing a new measurement; in the Gaussian case the corrected variance is always smaller, and in general it is smaller on average.

## Prediction and correction update the state's distribution

Everything the filter needs from the past observations is summarised in the one distribution  $P(x_t | Y_t)$ . It can therefore run online: it keeps that distribution, updates it when a frame arrives, and does the same amount of work at every frame however long the sequence has been.

The figure, from the course notes, draws the distribution of the airplane's position at the three stages of one step: the estimate at time  $t$ , the prediction for  $t + 1$ , and the correction at  $t + 1$ .

## Prediction and correction update the state's distribution



**Figure 4:** The filter updating the distribution of the airplane's position (Yuille and Kersten, 2025, Figure 4, second part, with the rows labelled). In each row the black curve is the airplane's path, the red airplane its true position and the blue one the observation. The size of the circle shows the spread of the distribution, in the colours of the original: red for the estimate at time  $t$ , blue for the prediction for  $t + 1$ , which is wider, and green for the correction at  $t + 1$ , which is narrower. The curve beside each row is the same distribution drawn in one dimension.

## Prediction and correction follow from the directed graph

Both steps are consequences of the two conditional independences of the graph. For the prediction, write the distribution of  $x_{t+1}$  as a marginal over the current state and factor the joint distribution,

$$P(x_{t+1} | Y_t) = \int dx_t P(x_{t+1}, x_t | Y_t) = \int dx_t P(x_{t+1} | x_t, Y_t) P(x_t | Y_t),$$

and  $P(x_{t+1} | x_t, Y_t) = P(x_{t+1} | x_t)$  because the next state is independent of the observations once the present state is given.

## Prediction and correction follow from the directed graph

For the correction, apply Bayes' rule to  $x_{t+1}$  and  $y_{t+1}$  with everything conditioned on  $Y_t$ ,

$$P(x_{t+1} | y_{t+1}, Y_t) = \frac{P(y_{t+1} | x_{t+1}, Y_t) P(x_{t+1} | Y_t)}{P(y_{t+1} | Y_t)},$$

and  $P(y_{t+1} | x_{t+1}, Y_t) = P(y_{t+1} | x_{t+1})$  because the observation depends only on its own state.

These two conditional independences are the only assumptions, so the recursion holds for any motion and observation models, Gaussian or not. What can be hard is to compute the integral and to represent the distributions, which may have any shape.

## With Gaussians the filter reduces to the Kalman equations

In general the integral of the prediction and the normalisation of the correction have no closed form, and the distributions have to be approximated. There is an important special case in which the filter reduces to algebraic equations (Kalman, 1960): the prior, the motion model and the observation model are all Gaussian,

$$\begin{aligned}\text{observation: } P(y_t | x_t) &= \mathcal{N}(x_t, \sigma_m^2), & \text{prior: } P(x_1) &= \mathcal{N}(\mu_0, \sigma_0^2), \\ \text{motion: } P(x_{t+1} | x_t) &= \mathcal{N}(x_t + \mu, \sigma_p^2).\end{aligned}$$

Here  $\mathcal{N}(m, s^2)$  is a Gaussian with mean  $m$  and variance  $s^2$ . The observation is the true position plus noise of variance  $\sigma_m^2$ ; the object moves on average by the distance  $\mu$  from  $t$  to  $t + 1$ , with variance  $\sigma_p^2$ ; and  $\mu_0$  is the mean position at the start, with variance  $\sigma_0^2$ .

The letter  $\mu$  without a subscript is the mean step. With a subscript,  $\mu_t$  is the mean of the estimate at time  $t$ , a different quantity.

## With Gaussians the filter reduces to the Kalman equations

If the estimate at time  $t$  is Gaussian,  $P(x_t | Y_t) = \mathcal{N}(\mu_t, \sigma_t^2)$ , then the prediction and the correction are Gaussian too:

prediction:  $P(x_{t+1} | Y_t) = \mathcal{N}(\mu + \mu_t, \sigma_p^2 + \sigma_t^2),$

correction:  $\mu_{t+1} = \mu + \mu_t - \frac{(\sigma_p^2 + \sigma_t^2)\{(\mu + \mu_t) - y_{t+1}\}}{\sigma_m^2 + (\sigma_p^2 + \sigma_t^2)},$

$$\sigma_{t+1}^2 = \frac{\sigma_m^2(\sigma_p^2 + \sigma_t^2)}{\sigma_m^2 + (\sigma_p^2 + \sigma_t^2)}.$$

Since the prior is Gaussian, every distribution the filter computes is Gaussian, and the filter only has to update two numbers at each step, a mean and a variance.

The prediction adds the mean step  $\mu$  to the mean and the motion variance  $\sigma_p^2$  to the variance. The correction moves the mean from the predicted position  $\mu + \mu_t$  towards the new observation, by a fraction of the difference between them, and makes the variance smaller than both the predicted variance  $\sigma_p^2 + \sigma_t^2$  and the observation variance  $\sigma_m^2$ .

## Deriving the Kalman equations

The prediction is the distribution of  $x_{t+1} = x_t + d$ , where  $x_t$  has the distribution  $\mathcal{N}(\mu_t, \sigma_t^2)$  given  $Y_t$  and the step  $d$  has the distribution  $\mathcal{N}(\mu, \sigma_p^2)$ , independent of  $x_t$ . A sum of two independent Gaussian variables is Gaussian, with the means added and the variances added, which gives  $\mathcal{N}(\mu + \mu_t, \sigma_p^2 + \sigma_t^2)$ .

For the correction write  $\bar{m} = \mu + \mu_t$  and  $v = \sigma_p^2 + \sigma_t^2$  for the mean and the variance of the prediction, and  $x, y$  for  $x_{t+1}, y_{t+1}$ . Up to factors that do not depend on  $x$ , the numerator of Bayes' rule is

$$\begin{aligned} & \exp\left\{-\frac{(y-x)^2}{2\sigma_m^2}\right\} \exp\left\{-\frac{(x-\bar{m})^2}{2v}\right\} \\ &= \exp\left\{-\frac{1}{2}\left(\frac{1}{\sigma_m^2} + \frac{1}{v}\right)x^2 + \left(\frac{y}{\sigma_m^2} + \frac{\bar{m}}{v}\right)x + \text{const}\right\}. \end{aligned}$$

The exponent is a quadratic in  $x$ , so the posterior is Gaussian. Matching it with the exponent  $-(x - \mu_{t+1})^2/(2\sigma_{t+1}^2)$  of a Gaussian gives

$$\frac{1}{\sigma_{t+1}^2} = \frac{1}{\sigma_m^2} + \frac{1}{v}, \quad \mu_{t+1} = \sigma_{t+1}^2 \left( \frac{y}{\sigma_m^2} + \frac{\bar{m}}{v} \right).$$

## Deriving the Kalman equations

Multiplying out,

$$\sigma_{t+1}^2 = \frac{\sigma_m^2 v}{\sigma_m^2 + v}, \quad \mu_{t+1} = \frac{\sigma_m^2 \bar{m} + v y}{\sigma_m^2 + v} = \bar{m} - \frac{v(\bar{m} - y)}{\sigma_m^2 + v},$$

which with  $\bar{m} = \mu + \mu_t$  and  $v = \sigma_p^2 + \sigma_t^2$  are the Kalman equations.

The initialisation is the same computation with the prior  $\mathcal{N}(\mu_0, \sigma_0^2)$  in the place of the prediction,

$$\mu_1 = \frac{\sigma_m^2 \mu_0 + \sigma_0^2 y_1}{\sigma_m^2 + \sigma_0^2}, \quad \sigma_1^2 = \frac{\sigma_m^2 \sigma_0^2}{\sigma_m^2 + \sigma_0^2}.$$

# The Kalman update is a weighted average of two cues

Rearranging the correction shows what it does:

$$\mu_{t+1} = \frac{\sigma_m^2}{\sigma_m^2 + \sigma_p^2 + \sigma_t^2} (\mu + \mu_t) + \frac{\sigma_p^2 + \sigma_t^2}{\sigma_m^2 + \sigma_p^2 + \sigma_t^2} y_{t+1},$$

$$\frac{1}{\sigma_{t+1}^2} = \frac{1}{\sigma_p^2 + \sigma_t^2} + \frac{1}{\sigma_m^2}.$$

The new estimate is a weighted average of the predicted position  $\mu + \mu_t$  and the observation  $y_{t+1}$ . The two weights add to one, and each source is weighted by the variance of the other: a precise observation, with small  $\sigma_m^2$ , pulls the estimate towards  $y_{t+1}$ , and a precise prediction pulls it towards  $\mu + \mu_t$ . The inverse variances, called precisions, add.

## The Kalman update is a weighted average of two cues

This is the linear rule for combining two cues from the lecture on cue combination. Two independent Gaussian measurements of one quantity are combined by weighting each with its reliability, the inverse of its variance, and the precision of the combination is the sum of the two precisions. The Kalman update combines the different sources of information in the same way. The prediction is one cue, and it carries everything learnt from the past observations; the new observation is the other.

## The Kalman update is a weighted average of two cues

The same update can be written as the prediction plus a correction term,

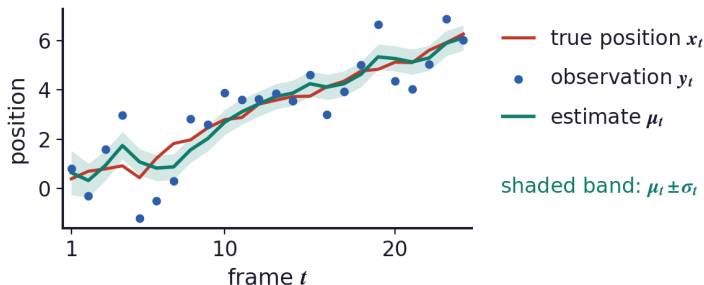
$$\mu_{t+1} = (\mu + \mu_t) + K_{t+1}\{y_{t+1} - (\mu + \mu_t)\}, \quad K_{t+1} = \frac{\sigma_p^2 + \sigma_t^2}{\sigma_m^2 + \sigma_p^2 + \sigma_t^2}.$$

The difference  $y_{t+1} - (\mu + \mu_t)$  is the prediction error, also called the innovation, the part of the observation that the prediction did not anticipate, and the factor  $K_{t+1}$ , between 0 and 1, is called the Kalman gain. If the object is at the mean predicted position,  $y_{t+1} = \mu + \mu_t$ , the correction term is zero and the estimate is the prediction.

In the example of the figure a target moves along a line by a mean step  $\mu = 0.25$  per frame, with motion noise  $\sigma_p = 0.3$ , and its position is observed at every frame with noise  $\sigma_m = 1$ ; the filter starts from the prior  $\mathcal{N}(0, 2^2)$ . Over 24 frames the root-mean-square (RMS) error of the observations is 1.02, and that of the estimate, from frame 4 on, when the prior has stopped mattering, is 0.39. The posterior standard deviation  $\sigma_t$  is 0.89 after the first observation, then 0.69, 0.60 and 0.56, and settles at 0.51.

# The Kalman update is a weighted average of two cues

**the estimate is smoother than the observations**



**Figure 5:** The Bayes-Kalman filter tracking a target that moves by a mean step of 0.25 per frame. The red line is the true position  $x_t$ , the blue dots are the observations  $y_t$ , the teal line is the estimate  $\mu_t$ , and the shaded band is  $\mu_t \pm \sigma_t$ . The estimate is smoother than the observations and follows the true position.

## The variance settles to a fixed value

The variance recursion contains only the three variances  $\sigma_m^2$ ,  $\sigma_p^2$  and  $\sigma_t^2$ , and none of the observations, so the whole sequence  $\sigma_1^2, \sigma_2^2, \dots$  is known before any data arrive. It converges to the fixed point  $\sigma^2$  of the recursion,

$$\sigma^2 = \frac{\sigma_m^2(\sigma_p^2 + \sigma^2)}{\sigma_m^2 + \sigma_p^2 + \sigma^2}.$$

Clearing the fraction gives the quadratic  $\sigma^4 + \sigma_p^2 \sigma^2 - \sigma_m^2 \sigma_p^2 = 0$ , whose positive root is

$$\sigma^2 = \frac{1}{2} \left( -\sigma_p^2 + \sqrt{\sigma_p^4 + 4 \sigma_m^2 \sigma_p^2} \right).$$

For  $\sigma_p = 0.3$  and  $\sigma_m = 1$  this is  $\sigma^2 = 0.258$ , so  $\sigma = 0.51$ , the value the example settles at. The gain then settles too, at  $K = 0.26$ : each new observation moves the estimate about a quarter of the way from the prediction to the observation.

The balance is between the two variances. The prediction adds  $\sigma_p^2$  at every step and the correction removes a fraction of what is there; the fixed point is where the two cancel.

## Special cases show what each variance does

Special cases of the update show what each of the three variances does.

If the observations are noiseless,  $\sigma_m = 0$ , the weight of the prediction is zero and  $\mu_{t+1} = y_{t+1}$ : forget the history and use the current observation as the estimate of  $x_{t+1}$ .

If the motion is perfectly predictable,  $\sigma_p = 0$ , the object moves by exactly  $\mu$  at each step and

$$\mu_{t+1} = \frac{\sigma_t^2 y_{t+1} + \sigma_m^2 (\mu + \mu_t)}{\sigma_m^2 + \sigma_t^2}, \quad \sigma_{t+1}^2 = \frac{\sigma_m^2 \sigma_t^2}{\sigma_m^2 + \sigma_t^2}.$$

The estimate is a weighted average of the observation and the predicted position, each weighted by the variance of the other. The variance of the prediction is now just  $\sigma_t^2$ , the uncertainty the estimate already had.

## Special cases show what each variance does

If in addition the object does not move,  $\mu = 0$ , then

$$\mu_{t+1} = \frac{\sigma_t^2 y_{t+1} + \sigma_m^2 \mu_t}{\sigma_m^2 + \sigma_t^2},$$

which is an online method for computing the maximum a posteriori (MAP) estimate of a static object at position  $x$ : the peak of the posterior of  $x$  given all the measurements, computed one measurement at a time. With the prior  $\mathcal{N}(\mu_0, \sigma_0^2)$ , the posterior after  $t$  measurements, computed in one batch, is Gaussian with

$$\frac{1}{\sigma_t^2} = \frac{1}{\sigma_0^2} + \frac{t}{\sigma_m^2}, \quad \mu_t = \sigma_t^2 \left( \frac{\mu_0}{\sigma_0^2} + \frac{1}{\sigma_m^2} \sum_{s=1}^t y_s \right),$$

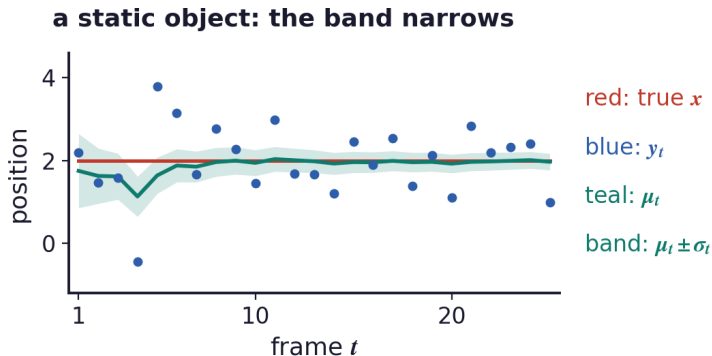
and the recursion reproduces it exactly at every  $t$ . Every measurement adds the same precision  $1/\sigma_m^2$ , so the variance falls like  $\sigma_m^2/t$  and the standard deviation like  $\sigma_m/\sqrt{t}$ : a static object is located more and more precisely.

## Special cases show what each variance does

In the figure the object is fixed at  $x = 2$ , the measurements have noise  $\sigma_m = 1$ , and the prior is  $\mathcal{N}(0, 2^2)$ . The posterior standard deviation falls from 0.89 after the first measurement to 0.20 after 25, and the final estimate is 1.97. The average of the 25 measurements is 1.99; the estimate is pulled slightly towards the prior mean 0.

The prior's precision  $1/\sigma_0^2 = 0.25$  is a quarter of the precision of one measurement, so the prior counts as a quarter of an observation. Early on it pulls the estimate towards 0, and after a few measurements it hardly matters, as in the last lecture, where a prior mattered most when there were few datapoints.

## Special cases show what each variance does



**Figure 6:** The filter for a static object at  $x = 2$ . The red line is the true position, the blue dots are the measurements  $y_t$ , the teal line is the estimate  $\mu_t$ , and the shaded band is  $\mu_t \pm \sigma_t$ , which narrows as the measurements accumulate.

## Bayes-Kalman extends to actions: control and world models

Bayes-Kalman is a directed graphical model for integrating information over time, and it can be used for visual tasks like tracking objects over time. It is part of the framework of Bayes decision theory (BDT): the filter supplies the posterior  $P(x_t | Y_t)$ , and a decision, such as where to point a camera or how to steer, is taken by minimising the expected loss under that posterior.

BDT can be extended to control theory, where the system makes a series of decisions over time and each decision changes the state it will observe next. Write  $u_t$  for the action taken at time  $t$ ;  $u$  is the usual letter in control, and  $a$  is kept for the transition probabilities of the hidden Markov models. The action enters the motion model,

$$\text{motion: } P(x_{t+1} | x_t, u_t), \quad \text{observation: } P(y_t | x_t),$$

and the graph gains one node per time, with an arrow from  $u_t$  into  $x_{t+1}$ .

## Bayes-Kalman extends to actions: control and world models

The filter carries the actions along. Write  $U_t = \{u_t, \dots, u_1\}$  for the actions taken so far. The prediction becomes

$$P(x_{t+1} | Y_t, U_t) = \int dx_t P(x_{t+1} | x_t, u_t) P(x_t | Y_t, U_{t-1}),$$

and the correction is unchanged, because the observation still depends only on the state. The actions are known to the system, since it chose them, so they enter the recursion as given quantities.

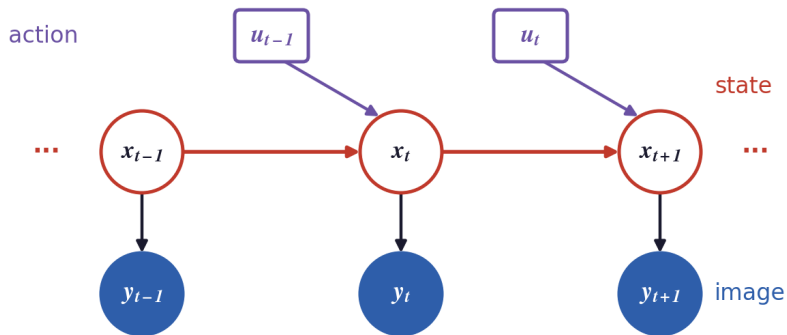
Dickmanns's group used this framework for automated cars. The state is the position of the car on the road and the positions of the other vehicles, estimated over time from the cameras, and the control keeps the car in its lane, avoids the other cars, and drives it from one place to another. In October 1994 their two vehicles drove more than 1,000 km in normal traffic on a multi-lane highway near Paris, largely autonomously (Dickmanns, 2002).

## Bayes-Kalman extends to actions: control and world models

World models extend these ideas by putting probability distributions on the images themselves. A world model generates the next image conditioned on the previous images and on an action, so a system that has one can imagine what it would see if it acted, before acting. The Generative World Explorer (Lu, Shu, Yuille, Khashabi and Chen, 2025) lets an agent imagine moving through a three-dimensional (3D) scene from the view at its current position: it generates the views the agent would see, and the agent uses these imagined views to update its beliefs about the scene and to make decisions.

The physical world models of Jiajun Wu (Wu, lecture slides) keep the structure of the figure: a hidden state that moves forward in time under an action, and an image generated from each state. What changes is how rich the state is and how the two arrows are computed.

## Bayes-Kalman extends to actions: control and world models



**Figure 7:** The Bayes-Kalman graph with actions. The states  $x_t$  (red) form a chain, each observation  $y_t$  (blue) depends on its state, and each action  $u_t$  (purple) is a parent of the next state, so the motion model becomes  $P(x_{t+1} | x_t, u_t)$ .

## Humans recover rich physical information from a video

Wu's lectures on physical scene understanding open with a short video of a chain reaction built from wooden blocks, ramps, balls and rows of dominoes: balls roll down ramps, dominoes topple one another, and each event sets off the next. From about ten seconds of such a video humans recover rich information:

- ▶ the geometry of the scene, meaning the 3D shapes of the objects;
- ▶ the physics of the objects, properties invisible in any single image, such as their masses and coefficients of friction;
- ▶ the physical events, the interactions between objects, such as collisions and rolling.

Humans also generalise from what they see. They easily answer questions like: what will happen next? what if the ball were heavier? how could the last domino be made to fall? The answers go beyond the images: they need a model of how the scene would evolve under conditions that were never shown.

## Humans recover rich physical information from a video

The video is a sequence of images of a world state that changes by physics. In the language of the Bayes-Kalman filter the world state is the hidden state  $x_t$ , physics is the motion model, and each frame of the video is an observation  $y_t$ . The state now holds far more than one position: every object, with its shape, mass and velocity. Inferring it is what answering the what-if questions requires.

In the figure the first image is a frame of the chain-reaction video, and the other two are close-ups of two kinds of event in it: a row of dominoes knocked over by collisions, and a ball rolling down a ramp.

# Humans recover rich physical information from a video



a frame of the video

collisions

rolling

**Figure 8:** A frame of the chain-reaction video, and close-ups of two kinds of physical event in it, collisions and rolling (images from Wu's lecture slides).

# Modeling the physical world: physics, graphics and actions

Wu's model of the physical world makes this precise. The world state at time  $t$  holds everything needed to describe the scene:

- ▶ the object intrinsics, their geometry and their physical properties, such as mass and friction;
- ▶ the object extrinsics, their positions and velocities;
- ▶ the scene description, the lighting and the camera parameters.

Graphics maps the world state to the image: a renderer, given the shapes, positions, lighting and camera, computes the image that would be seen. Physics maps the world state at time  $t - 1$ , together with the action taken at time  $t - 1$ , to the world state at time  $t$ : a physics engine simulates the forces and the collisions.

## Modeling the physical world: physics, graphics and actions

This is the Bayes-Kalman graph with actions. Physics is the motion model  $P(x_t | x_{t-1}, u_{t-1})$ , graphics is the observation model  $P(y_t | x_t)$ , and the action enters physics as the control input entered the motion of the cars. A simulator is deterministic; the probabilities come from the uncertainty about its inputs, such as a mass that cannot be seen, and from the noise between the rendered and the real image.

Inverse graphics is the inference in the other direction, from the images to the world state: the posterior of the world state given the images. The filter computes it by correcting its physical prediction with each new image.

In the figure the top row shows the world states at three times, drawn as wireframes of a row of dominoes, and the bottom row the three images. An action pushes the first domino; physics carries each world state to the next, and at time  $t + 1$  the dominoes have fallen.

# Modeling the physical world: physics, graphics and actions

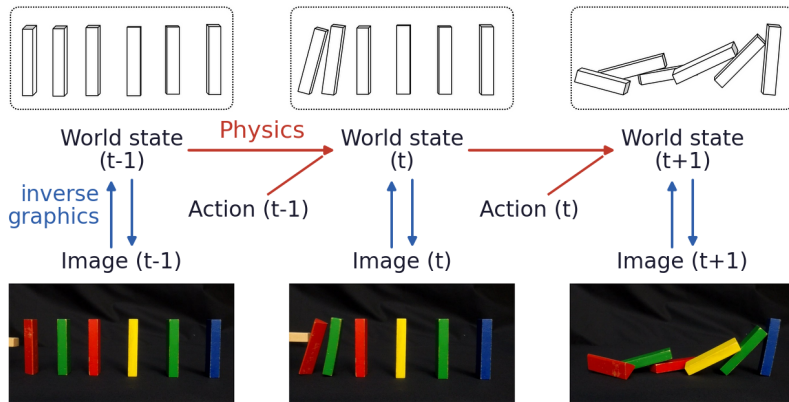


Figure 9: Wu's diagram of the physical world, redrawn around his six pictures so that the labels can be read at this size. The red arrows are physics, which takes a world state and an action to the next world state; the blue arrows between each world state and its image are graphics, pointing down, and inverse graphics, pointing up.

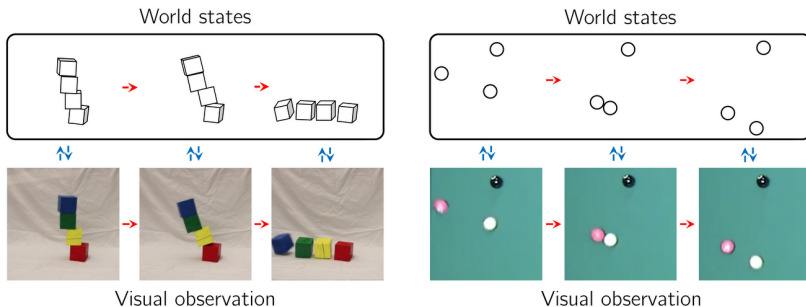
## Physical world representations are universal

The same kind of world state describes very different scenes. A falling tower of blocks and billiard balls on a table are both a few objects, each with a shape, a position and a velocity, evolving under the same physics: gravity, contact and collisions. What changes from one scene to the other is the list of objects and their properties; the structure of the model stays the same.

Because the laws are shared, one physics engine and one graphics engine serve both scenes, and only the objects in the world state change. Physics predicts the next world state, and graphics predicts the image seen at that time.

In the figure each box holds the world states at three times, drawn as outlines of the objects: a tower of four cubes falling, and balls on a table moving and colliding. The small red arrows between them are physics, the blue double arrows below are graphics and inverse graphics, and the pictures are the visual observations, with red arrows marking the same physics steps in the images.

# Physical world representations are universal



**Figure 10:** Wu's two examples of a world state: a tower of blocks and billiard balls on a table. In each, the box holds the world states at three times, the red arrows between them are physics, the blue double arrows are graphics and inverse graphics, and the photographs below are the visual observations, with red arrows for the same physics steps.

# Graphical models versus end-to-end deep learning

There are two broad ways to build such models. Approach 1 writes the causal structure down explicitly, and it has two families.

- ▶ Classical estimation and control, and graphical models such as hidden Markov models and Bayes nets. Their strength is that they are optimised for particular inference and learning algorithms, the Bayes-Kalman filter being one. Their weakness is limited expressiveness: the distributions must stay simple, often Gaussian or discrete, for inference to be efficient.
- ▶ Simulation engines for graphics and physics, and probabilistic programs, which are programs that make random choices, so that running them defines a probability distribution. Their strength is flexible, rich representations: they can render any scene and simulate any physics. Their weakness is that they lack efficient, general-purpose algorithms for inference and learning.

## Graphical models versus end-to-end deep learning

Approach II is end-to-end deep learning: a neural network is trained to map images directly to the quantities of interest, with no model of physics or graphics in between. Wu's examples include image classification, 3D shape reconstruction, the prediction of future states, such as whether a tower of blocks will fall, and actions learnt from the robot's own trials, such as poking and pushing objects or flying a drone.

Its strengths are efficient inference, one pass through the network with no model of the scene, and performance that scales with large labelled datasets. Its weakness is generalisation: such networks are limited outside the conditions of their training data.

## Graphical models versus end-to-end deep learning

Wu proposes to combine the two by leveraging the causal structure. The key idea is conditional independence: the graph says which variables depend directly on which, and so gives guidance on combining neural networks with simulation engines. It answers four questions:

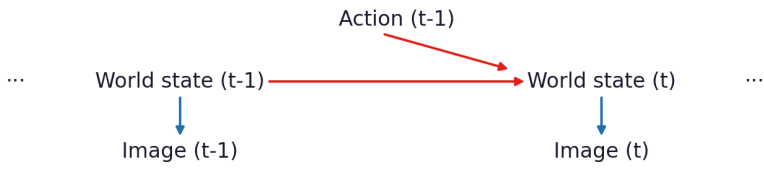
- ▶ when and where to use a simulation engine, and where a network;
- ▶ what training targets to use for the networks;
- ▶ what intermediate representations to use inside them;
- ▶ what training data to use for them.

In the graph of approach I, for example, the image depends on the past only through the current world state. A network that recognises the world state from one image can therefore be trained on single images, and the prediction of the future can be left to the physics engine.

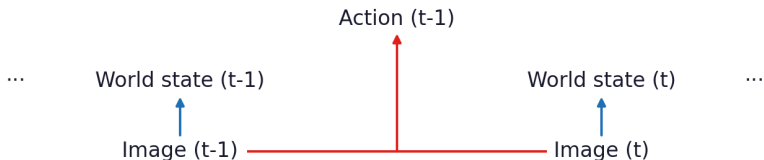
The figure redraws the two causal diagrams. In approach I the world states form a chain driven by an action (red), and each generates its image (blue). In approach II networks map each image up to a world state (blue), and map the images at  $t - 1$  and  $t$  directly to the action taken between them (red).

# Graphical models versus end-to-end deep learning

## Approach I: graphical models



## Approach II: end-to-end deep learning



**Figure 11:** The two approaches as causal diagrams, redrawn from Wu's slides at a legible size with the same nodes, arrows and colours. Approach I: physics (red) moves the world state forward under an action, and graphics (blue) generates each image. Approach II: networks map images to world states and to actions.

# Visual de-animation: inverse graphics, physics and graphics

Visual de-animation (VDA; Wu, Lu, Kohli, Freeman and Tenenbaum, 2017) is an example of the combination, with three parts.

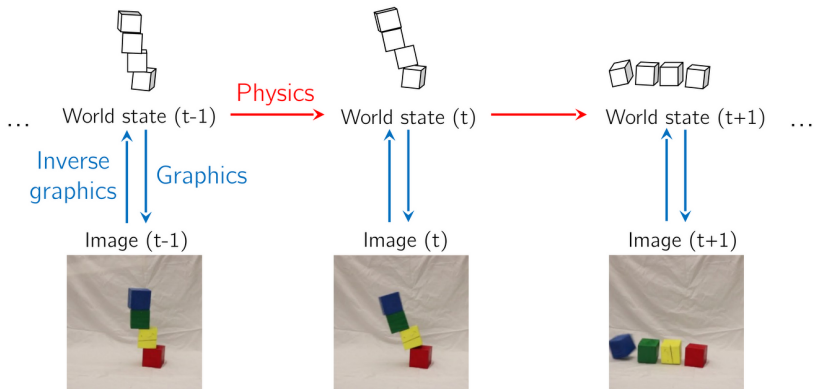
- ▶ A perception network inverts graphics: from each frame it estimates the world state, the shapes, positions and physical properties of the objects.
- ▶ A physics engine takes the estimated world state and predicts the future world states.
- ▶ A graphics engine renders the world states as images.

The name comes from animation, which goes from a description of a scene to images; de-animation goes back from the images to the description.

## Visual de-animation: inverse graphics, physics and graphics

Because the physics and graphics engines are given, the network can be trained with little supervision. The world state it estimates is rendered and compared with the input frame, and the simulated future is compared with the later frames, so much of the training signal comes from the video itself. Combining forward simulation engines with deep recognition networks allows learning with little or no supervision and offers rich generalisation power.

# Visual de-animation: inverse graphics, physics and graphics



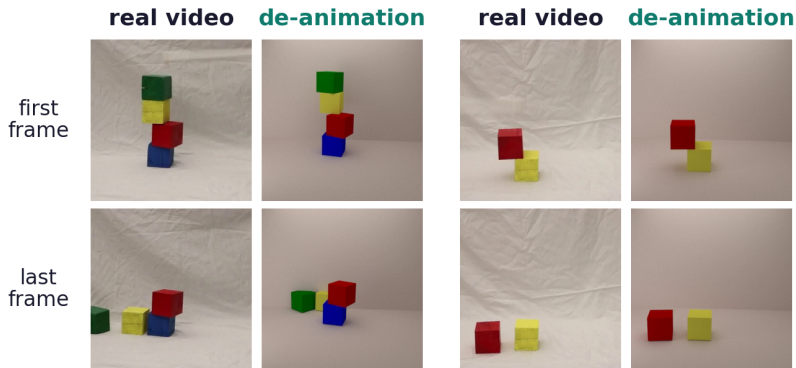
**Figure 12:** Visual de-animation (Wu et al., 2017; slide from Wu's lectures). Inverse graphics (blue, up) estimates the world state from each image; physics (red) predicts the next world state; graphics (blue, down) renders a world state as an image.

# Visual de-animation reconstructs and predicts real videos

The system was tested on real videos of towers of blocks. Each pair of columns in the figure is a real video and the rendering of its de-animation: the world state that the network estimates from the real video, rendered back by the graphics engine. The upper row shows the first frames and the lower row the last frames. The first frame gives the world state, and the physics engine predicts whether and how the tower falls.

In the first pair a tower of four blocks has its two upper blocks, green and yellow, overhanging the ones below; in the real video they topple to the left, and in the rendered de-animation they topple to the left as well. In the second pair a red block rests off-centre on a yellow one, and it falls to the left in both.

# Visual de-animation reconstructs and predicts real videos



**Figure 13:** Two real videos of block towers, each with the rendering of its de-animation (Wu et al., 2017; videos from Wu's slides). The upper row is the first frame of each video and the lower row the last frame.

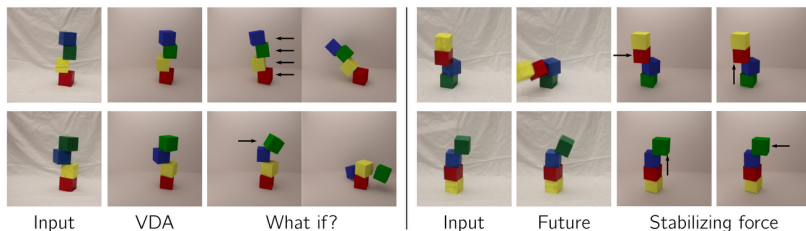
## De-animation answers what-if questions about the future

Because the world state is explicit, the system can be asked questions that go beyond the input image. The figure shows two kinds of generalisation, each on two examples, one per row.

- ▶ In the first group, Input is an image of a tower; VDA is the tower rendered from the world state that the network estimates from the input image; What if? shows forces applied to the estimated tower, drawn as arrows, and the predicted outcome.
- ▶ In the second group, Input is again an image; Future is the predicted fall of the tower; the two Stabilizing force panels each show, as an arrow, a force applied to keep the tower up.

The network recovers the world state once, and the physics engine can then be run under conditions that never occurred in any training video, such as an applied force. These are the what-if and how-to questions of the chain-reaction video, answered by simulation.

# De-animation answers what-if questions about the future



**Figure 14:** Generalisation by visual de-animation (Wu et al., 2017; slide from Wu's lectures). First group: an input image, its de-animation, and the outcome of the forces drawn as arrows. Second group: an input image, the predicted future, and two forces that each keep the tower standing.

## Dynamic programming exploits the linear structure

The Bayes-Kalman filter gives the distribution of each state given the observations. A different question is to find the best whole sequence of states, the one that explains all the observations together. That is a search over a chain, and a chain has a linear structure that dynamic programming (DP; Bellman, 1957) exploits. DP breaks a problem with linear structure into a sequence of stages, and solves each stage using the solutions of the stages before it.

The word programming in the name refers to planning, as in linear programming, from before the word came to mean writing computer code.

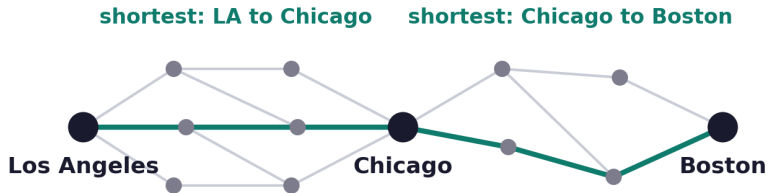
## Dynamic programming exploits the linear structure

The intuition is the shortest route. To find the shortest path from Los Angeles to Boston that goes via Chicago, it is only necessary to find the shortest path from Los Angeles to Chicago and the shortest path from Chicago to Boston, and to join them. Any other way of reaching Chicago is longer, and it is longer whatever happens after Chicago, so it cannot be part of the shortest route.

The same argument applies at every intermediate city, which is Bellman's principle of optimality: every part of an optimal path is itself optimal. When the route can pass through any of several cities at a stage, DP keeps, for each of them, the shortest path that reaches it, and extends these paths one stage at a time. The number of paths kept is the number of cities in a stage, however many stages there are.

In the figure the grey dots are intermediate cities and the grey lines candidate roads; the teal lines are the two shortest sub-paths, which together form the shortest route.

## Dynamic programming exploits the linear structure



**Figure 15:** A schematic of the route. Grey dots are intermediate cities and grey lines candidate roads. The teal lines are the shortest path from Los Angeles to Chicago and the shortest path from Chicago to Boston, which together are the shortest path from Los Angeles to Boston via Chicago.

## Minimise a sum of terms that couple neighbours on a chain

More technically, suppose we want to minimise

$$\varphi(x_1, \dots, x_N) = \varphi_{12}(x_1, x_2) + \varphi_{23}(x_2, x_3) + \dots + \varphi_{N-1,N}(x_{N-1}, x_N),$$

where each variable  $x_i$  takes one of  $k$  values,  $x_i \in \{1, \dots, k\}$ . Each term couples a pair of neighbours on the chain:  $\varphi_{i,i+1}(x_i, x_{i+1})$  is the cost of the pair of choices, such as the length of the road between two cities, or minus the logarithm of the probability of a transition. A term that depends on one variable only, such as the cost of a measurement at one pixel, can be added into one of the pair terms that contain that variable, so this form covers it.

A configuration is a choice of all  $N$  values. There are  $k^N$  configurations, so trying them all takes a time that grows exponentially with  $N$ . For  $k = 3$  values and  $N = 6$  variables there are  $3^6 = 729$ ; for a chain of 100 variables with 10 values each there are  $10^{100}$ .

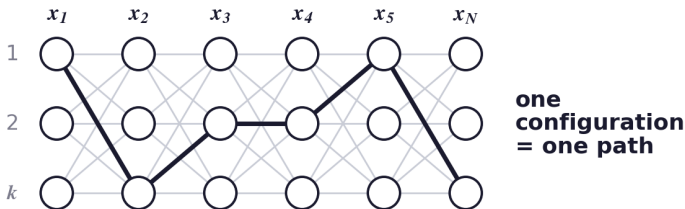
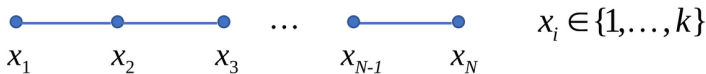
## Minimise a sum of terms that couple neighbours on a chain

The grid of values makes this concrete. Each column of the grid is one variable and each circle one of its  $k$  values. A configuration picks one circle in every column, which is a path through the grid from the first column to the last, and its cost is the sum of the costs of the edges it uses. Minimising  $\varphi$  is therefore finding the shortest path through the grid: the route problem again, with the columns as the stages and the values as the cities of a stage.

Stereo is an example from vision. For two cameras, the point in one image that matches a given point of the other lies on a line, the epipolar line, and the disparity is the shift between the two matching positions. Along one epipolar line the disparities of neighbouring pixels form a chain: the disparity of each pixel is one variable, the difference between the two images gives a matching cost for each disparity, and a term that couples neighbours favours disparities that change little along the line. Ohta and Kanade (1985) and Geiger, Ladendorf and Yuille (1995) found the best disparities along each line by DP.

# Minimise a sum of terms that couple neighbours on a chain

**the chain: each variable is coupled to its neighbours**



**Figure 16:** The chain of the course slides (Yuille, 2025), with the variables as blue dots, and the grid of its values for  $N = 6$  variables and  $k = 3$  values. Each column of the grid is one variable and each circle one of its values, with the last variable labelled  $x_N$  and the last row  $k$ . The black path is one configuration.

## The forward pass computes the shortest cost to each state

The forward pass computes, for each value of each variable, the cost of the shortest path that reaches it. Start with the second variable: for each value of  $x_2$  compute

$$h_2(x_2) = \min_{x_1} \varphi_{12}(x_1, x_2),$$

the shortest cost to  $x_2$ . This takes  $k$  comparisons for each of the  $k$  values of  $x_2$ . Next, for each value of  $x_3$ ,

$$\begin{aligned} h_3(x_3) &= \min_{x_1, x_2} \{ \varphi_{12}(x_1, x_2) + \varphi_{23}(x_2, x_3) \} \\ &= \min_{x_2} \{ h_2(x_2) + \varphi_{23}(x_2, x_3) \}. \end{aligned}$$

The second form is the clever bit. The minimum over  $x_1$  depends only on  $x_2$ , and it has already been computed and stored in  $h_2$ ; so the minimum over two variables becomes a minimum over one, and it again costs  $k^2$  operations.

## The forward pass computes the shortest cost to each state

In general,

$$h_m(x_m) = \min_{x_{m-1}} \{h_{m-1}(x_{m-1}) + \varphi_{m-1,m}(x_{m-1}, x_m)\},$$

and the table  $h_m$  stores, for each value of the current variable, the cost of the best path that reaches it. Along with each  $h_m(x_m)$  the forward pass stores the value of  $x_{m-1}$  that attains the minimum, the best way into  $x_m$ .

The forward pass ends with  $h_N(x_N)$ , the shortest cost to each value of the last variable, after  $N - 1$  steps of  $k^2$  operations each: a time polynomial in the number of nodes and in the number of values, where trying every configuration took a time exponential in the number of nodes.

## The forward recursion is exact on a chain

The recursion is exact, and the reason is the chain structure. By definition

$$h_m(x_m) = \min_{x_1, \dots, x_{m-1}} \sum_{i=1}^{m-1} \varphi_{i,i+1}(x_i, x_{i+1}).$$

Only the last term,  $\varphi_{m-1,m}$ , contains  $x_m$ , and it does not contain  $x_1, \dots, x_{m-2}$ . Split the minimum into a minimum over  $x_{m-1}$  outside and over  $x_1, \dots, x_{m-2}$  inside, and take the last term out of the inner minimum:

$$h_m(x_m) = \min_{x_{m-1}} \left\{ \min_{x_1, \dots, x_{m-2}} \sum_{i=1}^{m-2} \varphi_{i,i+1}(x_i, x_{i+1}) + \varphi_{m-1,m}(x_{m-1}, x_m) \right\}.$$

The inner minimum is  $h_{m-1}(x_{m-1})$ , which gives the recursion; for  $m = 2$  the inner sum is empty and  $h_1 = 0$ .

The step depends on the chain. On a graph with a closed loop some term would couple  $x_m$  to a variable earlier than  $x_{m-1}$ , that term could not be taken out, and the inner minimum would depend on  $x_m$  as well.

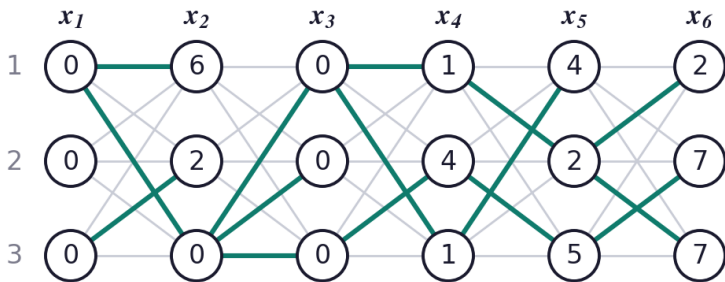
## The forward pass computes the shortest cost to each state

In the example of the figure  $N = 6$ ,  $k = 3$ , and the 45 costs  $\varphi_{i,i+1}(a, b)$  are integers drawn uniformly from 0 to 9. Listing the values 1, 2, 3 in order, the forward pass gives

$$h_2 = (6, 2, 0), \quad h_3 = (0, 0, 0), \quad h_4 = (1, 4, 1), \\ h_5 = (4, 2, 5), \quad h_6 = (2, 7, 7).$$

For every value of every variable after the first, the teal edge in the figure is the best way into it, the edge that attains the minimum in  $h$ . Two minima are ties, for value 1 of  $x_2$  and value 3 of  $x_5$ ; they are broken by taking the first index, and neither lies on the best path. The numbers in the circles of the figure are these values of  $h$ .

The forward pass computes the shortest cost to each state



**kept: the best way into each value**

**Figure 17:** The forward pass on an example with  $N = 6$  variables and  $k = 3$  values, with costs drawn at random. The number in each circle is  $h_m(x_m)$ , with  $h_1 = 0$ . For every value of every variable after the first, the teal edge is the best way into it: the edge from the previous variable that attains the minimum in  $h$ .

## The backward pass recovers the states of the shortest path

The forward pass gives the shortest cost,  $\min_{x_N} h_N(x_N)$ , and the backward pass recovers the states that give it. Start from the best final value and step back, choosing each value given the value after it:

$$\hat{x}_N = \arg \min_{x_N} h_N(x_N),$$

$$\hat{x}_{N-1} = \arg \min_{x_{N-1}} \{h_{N-1}(x_{N-1}) + \varphi_{N-1,N}(x_{N-1}, \hat{x}_N)\},$$

and so on down to  $\hat{x}_1$ . Each of these minimisations is one the forward pass already carried out, for the value chosen after it; if the forward pass stored the best way into each value, the backward pass simply follows the stored edges back from  $\hat{x}_N$ .

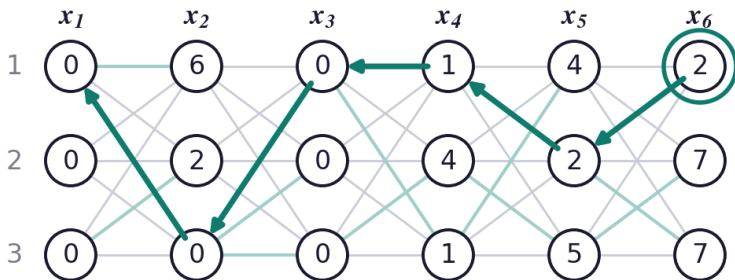
This recovers the states  $\hat{x}_N, \hat{x}_{N-1}, \dots, \hat{x}_1$  that give the shortest cost. The advantage is efficiency: about  $k^2 N$  operations in place of considering all the  $k^N$  configurations of  $\varphi(x_1, \dots, x_N)$ .

## The backward pass recovers the states of the shortest path

In the example the best final value is  $\hat{x}_6 = 1$ , with shortest cost  $h_6(1) = 2$ , and following the kept edges back gives the path 1, 3, 1, 1, 2, 1. A brute-force search over all  $3^6 = 729$  configurations gives the same minimum, 2.

With  $k = 3$  and  $N = 6$  the saving is modest:  $(N - 1)k^2 = 45$  operations against 729 configurations. For stereo, with tens of disparities and hundreds of pixels on each epipolar line, it is the difference between a computation of a fraction of a second and one that is impossible in practice.

The backward pass recovers the states of the shortest path



**follow the kept edges back**

**Figure 18:** The backward pass on the same example, with  $h_m(x_m)$  in each circle. The circled node is the best final value, and the teal arrows follow the kept edges back from it: the shortest path is 1, 3, 1, 1, 2, 1. The pale edges are the best ways into the other values, which the backward pass does not use.

## DP applies to any graph without closed loops

DP extends in three ways. First, DP can be applied to any graph without closed loops, such as a tree. A closed loop, or cycle, is a path along the edges that returns to its start without using an edge twice, and a tree is a connected graph without one. A tree is processed from the leaves inwards: a leaf is coupled to one neighbour only, so it can be minimised out, which leaves behind a function of that neighbour, exactly as  $h$  did on the chain. Repeating this removes the whole tree down to a single node.

Second, DP can be modified to compute other properties of interest, such as marginal probabilities. Replacing the minimum of summed costs by the sum of multiplied probabilities turns the forward pass into the computation of a marginal. For a chain distribution

$P(x_1, \dots, x_N) = P(x_1) \prod_{m=2}^N P(x_m | x_{m-1})$ , summing out  $x_1$ , then  $x_2$ , and so on, is the recursion

$$g_1(x_1) = P(x_1), \quad g_m(x_m) = \sum_{x_{m-1}} g_{m-1}(x_{m-1}) P(x_m | x_{m-1}),$$

which ends with  $g_N(x_N) = P(x_N)$  after about  $k^2 N$  operations.

## DP applies to any graph without closed loops

The example of the lecture on directed graphs is of this kind. A friend who claims psychic powers ( $x_3$ ) predicts that a coin will land heads ( $x_1$ ), which a two-headed coin ( $x_4$ ) would also explain, and a test of levitating a pencil ( $x_2$ ) depends on the psychic powers only. The graph  $P(x_1, x_2, x_3, x_4) = P(x_1 | x_3, x_4)P(x_2 | x_3)P(x_3)P(x_4)$  has edges  $x_3x_1$ ,  $x_4x_1$  and  $x_3x_2$ , which form a tree, and with binary variables

$$P(x_1 = 1) = \sum_{x_3, x_4} P(x_1 = 1 | x_3, x_4) P(x_3) P(x_4) \sum_{x_2} P(x_2 | x_3),$$

and the last sum is 1. What remains is a sum over the  $2^2 = 4$  values of  $(x_3, x_4)$ , where the sum over all three variables  $x_2, x_3, x_4$  had  $2^3 = 8$  terms.

Third, graphs with closed loops can be converted into a tree by grouping their variables into clusters, and DP then runs on the tree of clusters: this is the junction tree algorithm (Lauritzen and Spiegelhalter, 1988). The cost is the size of the clusters, since a cluster of  $n$  variables with  $k$  values each has  $k^n$  values of its own.

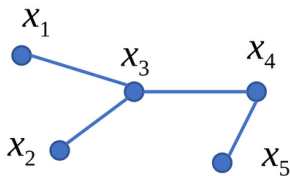
## DP applies to any graph without closed loops

For many graphs the tree is so large that computation on it is impractical. The image lattice is the standard example: every square of four neighbouring pixels is a closed loop, and the clusters needed to remove all the loops grow with the smaller side of the image, to about a whole row or column of pixels.

This is why the lectures on Markov random fields used approximate methods, such as mean field theory, in place of exact DP.

DP applies to any graph without closed loops

**a tree: DP applies**



**a lattice: closed loops**

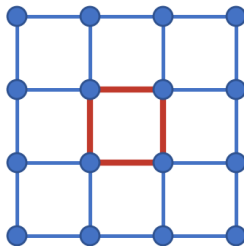


Figure 19: A tree, from the course slides (Yuille, 2025), on which DP applies, and a lattice, on which exact DP is impractical. The blue dots are variables and the lines their couplings; the red square is one of the closed loops of the lattice.

## A Markov model is a probability distribution for a sequence

A discrete Markov process is a probability distribution for a sequence. At each time  $t$  the system is in one of  $N$  distinct states  $s_1, \dots, s_N$ ; write  $q_t$  for the state at time  $t$ , so that  $q_t = s_i$  means the system is in state  $s_i$ . Here  $N$  is the number of states; in the chain of dynamic programming it was the number of variables. In general the next state could depend on the whole history, through  $P(q_{t+1} = s_j \mid q_t = s_i, q_{t-1} = s_k, \dots)$ . A first-order Markov model assumes

$$P(q_{t+1} = s_j \mid q_t = s_i, q_{t-1} = s_k, \dots) = P(q_{t+1} = s_j \mid q_t = s_i) :$$

given the present state, the future is independent of the past. A model of order two would let the next state depend on the last two states, and so on.

## A Markov model is a probability distribution for a sequence

The model is specified by two sets of numbers. The transition probabilities

$$a_{ij} = P(q_{t+1} = s_j \mid q_t = s_i), \quad a_{ij} \geq 0, \quad \sum_{j=1}^N a_{ij} = 1 \quad \text{for all } i,$$

are assumed independent of time; the conditions say that from any state the system goes to some state. The initial probabilities  $\pi_i = P(q_1 = s_i)$  give the distribution of the first state.

## A Markov model is a probability distribution for a sequence

The probability of a whole sequence of states is the product

$$P(q_1, \dots, q_T) = \pi_{q_1} a_{q_1 q_2} a_{q_2 q_3} \cdots a_{q_{T-1} q_T}.$$

This is the chain of states of the Bayes-Kalman graph, with discrete states in place of continuous ones and a table of transition probabilities in place of a Gaussian motion model.

## A Markov model is a probability distribution for a sequence

There are two cases: (I) observable Markov models, in which the states are seen, and (II) hidden Markov models (HMMs), in which they are not. Both are exponential distributions; for an HMM this holds for the joint distribution of its states and observations. Taking the logarithm of the product above,

$$\log P(q_1, \dots, q_T) = \sum_i n_i \log \pi_i + \sum_{i,j} n_{ij} \log a_{ij},$$

where  $n_i$  is 1 if the sequence starts in  $s_i$  and 0 otherwise, and  $n_{ij}$  is the number of transitions from  $s_i$  to  $s_j$  in the sequence. The distribution is the exponential of a sum of counts times log parameters, which is the form of an exponential distribution with the counts as its sufficient statistics, the only features of the data that the distribution depends on.

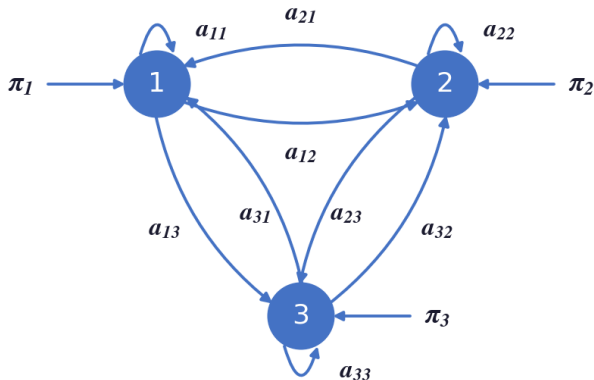
## A Markov model is a probability distribution for a sequence

The graph of a Markov model, and of an HMM, has no closed loops, so dynamic programming can be used for inference.

Hidden Markov models were the state of the art for speech recognition for many years (Rabiner, 1989), until deep networks replaced first their observation models and later the whole system. The hidden states are the parts of the words being spoken, which follow one another in a typical order, and the observations are short windows of the sound.

In the figure each circle is a state, each arrow from state  $i$  to state  $j$  carries its transition probability  $a_{ij}$ , including the loops  $a_{ii}$  that return to the same state, and each arrow coming into state  $i$  from outside carries the initial probability  $\pi_i$ .

A Markov model is a probability distribution for a sequence



**Figure 20:** A Markov model with three states, from the course slides (Yuille, 2025), redrawn so that the labels can be read, with the states in the blue of the original. The arrow from state  $i$  to state  $j$  is labelled with the transition probability  $a_{ij}$ , and the arrow into state  $i$  from outside with the initial probability  $\pi_i$ .

## An observable Markov model is learnt by counting

In an observable Markov model we can directly observe the states  $q_t$ , and this enables us to learn the transition probabilities. The observation sequence is the state sequence itself,  $O = Q = \{q_1, \dots, q_T\}$ , and its probability is

$$P(O = Q | A, \pi) = P(q_1) \prod_{t=2}^T P(q_t | q_{t-1}) = \pi_{q_1} a_{q_1 q_2} \cdots a_{q_{T-1} q_T},$$

the initial probability times  $T - 1$  transition probabilities, where  $A = \{a_{ij}\}$  is the table of transition probabilities.

## An observable Markov model is learnt by counting

The example of the course slides has urns with three kinds of ball,  $s_1 =$  red,  $s_2 =$  blue and  $s_3 =$  green. The state is the urn the ball is drawn from, and each urn holds balls of one colour, so the colour tells us which urn. With

$$\pi = (0.5, 0.2, 0.3), \quad A = \begin{pmatrix} 0.4 & 0.3 & 0.3 \\ 0.2 & 0.6 & 0.2 \\ 0.1 & 0.1 & 0.8 \end{pmatrix},$$

where row  $i$  lists the probabilities of moving from  $s_i$  to each state, the sequence  $O = \{s_1, s_1, s_3, s_3\}$  has probability

$$P(O \mid A, \pi) = \pi_1 a_{11} a_{13} a_{33} = 0.5 \times 0.4 \times 0.3 \times 0.8 = 0.048.$$

## An observable Markov model is learnt by counting

Learning is counting. Suppose we have  $K$  sequences of length  $T$ , and write  $q_t^k$  for the state at time  $t$  of the  $k$ -th sequence. The maximum likelihood (ML) estimates, the parameters under which the observed sequences are most probable,

$$(\hat{A}, \hat{\pi}) = \arg \max_{A, \pi} \prod_{k=1}^K P(O = Q_k \mid A, \pi),$$

are frequencies. The initial probability of  $s_i$  is estimated by the fraction of sequences that start in  $s_i$ ,

$$\hat{\pi}_i = \frac{\#[\text{sequences starting with } s_i]}{\#[\text{sequences}]} = \frac{1}{K} \sum_k I(q_1^k = s_i),$$

where the indicator  $I(\cdot)$  is 1 when its condition holds and 0 otherwise.

## An observable Markov model is learnt by counting

The transition probability from  $s_i$  to  $s_j$  is estimated by the fraction of the transitions out of  $s_i$  that go to  $s_j$ ,

$$\hat{a}_{ij} = \frac{\#[\text{transitions from } s_i \text{ to } s_j]}{\#[\text{transitions from } s_i]} = \frac{\sum_k \sum_{t=1}^{T-1} I(q_t^k = s_i, q_{t+1}^k = s_j)}{\sum_k \sum_{t=1}^{T-1} I(q_t^k = s_i)}.$$

For the urns,  $\hat{a}_{12}$  is the number of times a blue ball follows a red ball, divided by the number of red balls that have a successor.

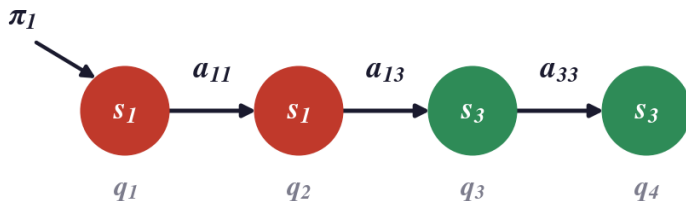
## An observable Markov model is learnt by counting

The counts are sufficient statistics for the Markov model: the likelihood depends on the data only through them, so two sets of sequences with the same counts give the same estimates. Maximum likelihood equates empirical statistics with model statistics. The fitted model expects each transition out of  $s_i$  in the proportion in which it was observed, the same condition as in the learning rule of the Boltzmann machine, where learning stops when the correlations under the data equal the correlations under the model.

In the figure the four coloured circles are the observed sequence  $\{s_1, s_1, s_3, s_3\}$ , red for  $s_1$  and green for  $s_3$ . Its probability is the initial probability  $\pi_1$  times the three transition probabilities written on the arrows.

An observable Markov model is learnt by counting

**$O = \{\text{red, red, green, green}\}$**



**Figure 21:** The sequence  $O = \{s_1, s_1, s_3, s_3\}$  of the urn example, with red for  $s_1$  and green for  $s_3$ . Its probability is the initial probability  $\pi_1$  times the transition probabilities  $a_{11}$ ,  $a_{13}$  and  $a_{33}$  on the arrows.

## Counting is maximum likelihood

The log-likelihood of the  $K$  sequences is

$$\log \prod_k P(O = Q_k | A, \pi) = \sum_i n_i \log \pi_i + \sum_{i,j} n_{ij} \log a_{ij},$$

where now  $n_i$  is the number of sequences that start in  $s_i$  and  $n_{ij}$  the number of transitions from  $s_i$  to  $s_j$  in all of them. The rows of  $A$  are separate problems: for each  $i$ , maximise  $\sum_j n_{ij} \log a_{ij}$  subject to  $\sum_j a_{ij} = 1$ . With a Lagrange multiplier  $\nu_i$ , an extra variable that enforces the constraint, set the derivative of

$$\sum_j n_{ij} \log a_{ij} - \nu_i \left( \sum_j a_{ij} - 1 \right)$$

with respect to  $a_{ij}$  to zero:  $n_{ij}/a_{ij} = \nu_i$ , so  $a_{ij}$  is proportional to  $n_{ij}$ , and the constraint fixes  $\nu_i = \sum_j n_{ij}$ . Therefore  $\hat{a}_{ij} = n_{ij}/\sum_j n_{ij}$ , and the same argument gives  $\hat{\pi}_i = n_i/K$ .

## In an HMM each hidden state emits an observation

In a hidden Markov model the states are hidden, and each state produces an observation that we see. The state  $q_t \in \{s_1, \dots, s_N\}$  follows a Markov chain as before. The observation  $O_t \in \{v_1, \dots, v_M\}$  is one of  $M$  symbols, and it depends only on the state at the same time, through the observation probabilities

$$b_j(m) = P(O_t = v_m \mid q_t = s_j),$$

the probability that we observe  $v_m$  if the state is  $s_j$ . The state is said to emit the observation.

There are two sources of stochasticity: the transition from one state to the next is random, with probabilities  $a_{ij}$ , and the observation from a state is random, with probabilities  $b_j(m)$ .

## In an HMM each hidden state emits an observation

In the urn example, let each urn now contain balls of different colours: urn 1 mostly red, urn 2 mostly blue and urn 3 mostly green. A ball is drawn from the current urn, its colour is recorded and it is put back, and the next urn is chosen with the transition probabilities. The observation is the colour of the ball, and we do not know which urn it comes from: the state is hidden.

## In an HMM each hidden state emits an observation

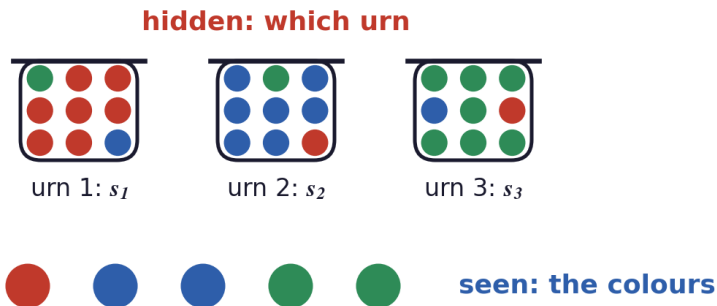
The HMM is the Bayes-Kalman graph with discrete states and discrete observations: the transition probabilities are its motion model and the observation probabilities its observation model. Its joint distribution is the product of one conditional per node,

$$P(O, Q | \lambda) = \pi_{q_1} b_{q_1}(O_1) \prod_{t=2}^T a_{q_{t-1}q_t} b_{q_t}(O_t),$$

where  $\lambda$  stands for all the parameters.

In the figure the three urns are the hidden states, each drawn with the mix of colours it holds, and the row of balls below is an observed sequence of draws. Which urn each ball came from is the hidden part.

In an HMM each hidden state emits an observation



**Figure 22:** The urn example of a hidden Markov model. Each urn is a state and holds balls of all three colours in different proportions: urn 1 mostly red, urn 2 mostly blue and urn 3 mostly green. A sequence of drawn balls is seen; which urn each came from is hidden.

## Three basic problems: evaluation, decoding and learning

An HMM has five elements:

1.  $N$ , the number of states, with states  $S = \{s_1, \dots, s_N\}$ ;
2.  $M$ , the number of observation symbols, with alphabet  $V = \{v_1, \dots, v_M\}$ ;
3. the state transition probabilities  $A = \{a_{ij}\}$ ,  
 $a_{ij} = P(q_{t+1} = s_j \mid q_t = s_i)$ ;
4. the observation probabilities  $B = \{b_j(m)\}$ ,  
 $b_j(m) = P(O_t = v_m \mid q_t = s_j)$ ;
5. the initial state probabilities  $\pi = \{\pi_i\}$ ,  $\pi_i = P(q_1 = s_i)$ .

The triple  $\lambda = (A, B, \pi)$  specifies the parameter set of an HMM.

# Three basic problems: evaluation, decoding and learning

There are three basic problems:

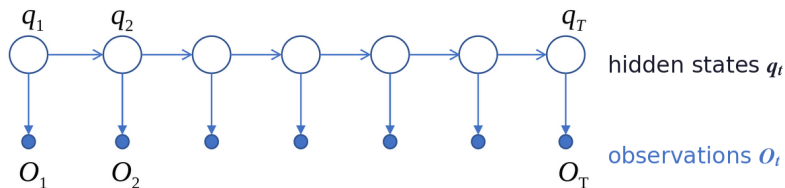
- (1) evaluation: given a model  $\lambda$ , evaluate the probability  $P(O | \lambda)$  of any observation sequence  $O = (O_1, \dots, O_T)$ ;
- (2) decoding: given  $\lambda$  and  $O$ , find the most probable state sequence  $Q = \{q_1, \dots, q_T\}$  given the observations,  
 $Q^* = \arg \max_Q P(Q | O, \lambda)$ ;
- (3) learning: given a training set of sequences  $X = \{O^k\}$ , find  
 $\lambda^* = \arg \max_{\lambda} P(X | \lambda)$ .

## Three basic problems: evaluation, decoding and learning

Each problem has its use. Evaluation compares models: the model that gives the data the higher probability explains them better, which is how an HMM recogniser decides which word was spoken. Decoding recovers the hidden story behind the observations, such as the sequence of urns, or of camera views in a sports video. Learning fits the model to data.

The first two problems are solved by DP on the chain of the figure, the third by EM, with DP inside the E-step. In the figure, from the course slides, the upper row is the hidden states  $q_1, \dots, q_T$  and each has its observation  $O_t$  below it.

## Three basic problems: evaluation, decoding and learning



**Figure 23:** The chain of an HMM, from the course slides (Yuille, 2025): the hidden states  $q_1, \dots, q_T$  as hollow circles, each with its observation  $O_t$  as a filled dot below it.

## Evaluation sums over every possible state sequence

Given an observation sequence  $O = (O_1, \dots, O_T)$  and a state sequence  $Q$ , the probability of observing  $O$  is

$$P(O | Q, \lambda) = \prod_{t=1}^T P(O_t | q_t, \lambda) = b_{q_1}(O_1) b_{q_2}(O_2) \cdots b_{q_T}(O_T),$$

because given the states each observation depends only on its own state. The state sequence  $Q$  is unknown. Its prior probability, the probability of the state sequence before any observation, is

$$P(Q | \lambda) = P(q_1) \prod_{t=2}^T P(q_t | q_{t-1}) = \pi_{q_1} a_{q_1 q_2} \cdots a_{q_{T-1} q_T}.$$

## Evaluation sums over every possible state sequence

The joint probability is the product of the two,

$$\begin{aligned} P(O, Q | \lambda) &= P(q_1) \prod_{t=2}^T P(q_t | q_{t-1}) \prod_{t=1}^T P(O_t | q_t) \\ &= \pi_{q_1} b_{q_1}(O_1) a_{q_1 q_2} b_{q_2}(O_2) \cdots a_{q_{T-1} q_T} b_{q_T}(O_T), \end{aligned}$$

and the probability of the observations is the sum over all state sequences,

$$P(O | \lambda) = \sum_Q P(O | Q, \lambda) P(Q | \lambda).$$

This summation is impractical directly, because there are  $N^T$  possible  $Q$ : each of the  $T$  times has  $N$  possible states. For the three urns and a sequence of 100 draws that is  $3^{100}$ , about  $5 \times 10^{47}$  state sequences.

## The forward algorithm computes $P(O | \lambda)$ by DP

There is an efficient procedure to calculate  $P(O | \lambda)$ , the forward-backward procedure, which is essentially DP and exploits the Markov structure of the distribution; its forward half computes  $P(O | \lambda)$ . Divide the sequence into two parts, the observations from 1 to  $t$  and those from  $t + 1$  to  $T$ . The forward variable

$$\alpha_t(i) = P(O_1, \dots, O_t, q_t = s_i | \lambda)$$

is the probability of observing the partial sequence up to time  $t$  and being in state  $s_i$  at time  $t$ :  $\alpha_t(i)$  explains the first  $t$  observations and ends in state  $s_i$ . It is computed recursively,

initialisation:  $\alpha_1(i) = P(O_1 | q_1 = s_i, \lambda) P(q_1 = s_i | \lambda) = \pi_i b_i(O_1)$ ,

$$\text{recursion: } \alpha_{t+1}(j) = \left[ \sum_{i=1}^N \alpha_t(i) a_{ij} \right] b_j(O_{t+1}),$$

$$\text{termination: } P(O | \lambda) = \sum_{i=1}^N P(O, q_T = s_i | \lambda) = \sum_{i=1}^N \alpha_T(i).$$

The forward algorithm computes  $P(O | \lambda)$  by DP

In the recursion the bracket sums over the states at time  $t$ , each weighted by its probability of moving to  $s_j$ , and the factor  $b_j(O_{t+1})$  multiplies in the probability of the next observation. Each step computes  $N$  sums of  $N$  terms, so computing all the  $\alpha_t(i)$  takes about  $N^2T$  operations, where the direct sum took a number exponential in  $T$ . This solves the first problem, computing the probability of generating the data given the model.

## The forward recursion follows from the Markov property

The recursion uses the same two conditional independences as the Bayes-Kalman filter. Writing  $O_{1:t}$  for  $O_1, \dots, O_t$  and leaving the  $\lambda$  out of the conditionals,

$$\begin{aligned}\alpha_{t+1}(j) &= \sum_{i=1}^N P(O_{1:t}, q_t = s_i, q_{t+1} = s_j, O_{t+1}) \\ &= \sum_{i=1}^N P(O_{1:t}, q_t = s_i) P(q_{t+1} = s_j \mid q_t = s_i, O_{1:t}) \\ &\quad \times P(O_{t+1} \mid q_{t+1} = s_j, q_t = s_i, O_{1:t}) \\ &= \sum_{i=1}^N \alpha_t(i) a_{ij} b_j(O_{t+1}).\end{aligned}$$

The first line sums over the state at time  $t$ . The second factors the joint probability. The third uses that the next state depends only on the present state, and that the observation depends only on its own state.

## The forward algorithm computes $P(O | \lambda)$ by DP

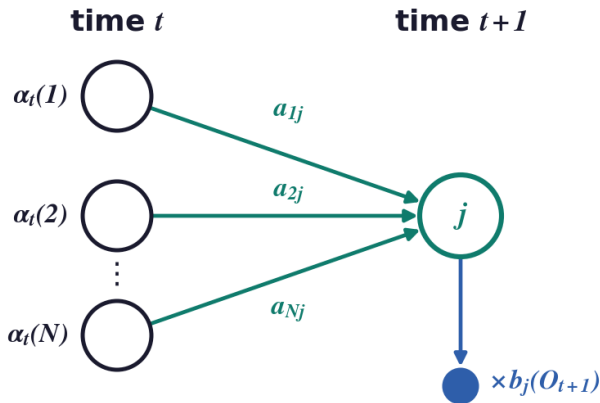
The forward algorithm is the forward pass of DP, with the minimum and the sum on costs replaced by the sum and the product on probabilities: the sum over  $i$  plays the part of the minimum over  $x_{m-1}$ , and multiplying by  $a_{ij}b_j(O_{t+1})$  plays the part of adding the cost  $\varphi_{m-1,m}$ .

It is also the Bayes-Kalman filter for a discrete state. The sum over  $i$  is the prediction, with the integral over  $x_t$  replaced by a sum over the states, and the factor  $b_j(O_{t+1})$  is the correction, Bayes' rule without its normalisation. Normalising gives the distribution the filter computes,

$$\frac{\alpha_t(i)}{\sum_j \alpha_t(j)} = P(q_t = s_i | O_1, \dots, O_t, \lambda).$$

In the figure,  $\alpha_{t+1}(j)$  collects  $\alpha_t(i)$  from every state at time  $t$  along the teal arrows, each multiplied by  $a_{ij}$ , and is then multiplied by  $b_j(O_{t+1})$ , the probability of the observation at  $t+1$  (blue).

The forward algorithm computes  $P(O | \lambda)$  by DP



**Figure 24:** One step of the forward recursion. The value  $\alpha_{t+1}(j)$  collects  $\alpha_t(i)$  from every state at time  $t$ , each multiplied by the transition probability  $a_{ij}$  (teal), and is then multiplied by the probability  $b_j(O_{t+1})$  of the observation at time  $t+1$  (blue). The dots stand for the states between  $s_2$  and  $s_N$ .

## The most probable state at each time can be inconsistent

A second recursion runs backwards in time. It is needed for the probability of each single state given all the observations, and so for greedy decoding and for learning. The backward variable

$$\beta_t(i) = P(O_{t+1}, \dots, O_T \mid q_t = s_i, \lambda)$$

is the probability of the remaining observations, from  $t + 1$  to  $T$ , given that the state at time  $t$  is  $s_i$ . It is computed from the end of the sequence,

$$\beta_T(i) = 1, \quad \beta_t(i) = \sum_{j=1}^N a_{ij} b_j(O_{t+1}) \beta_{t+1}(j).$$

At time  $T$  there are no remaining observations, so their probability is 1. Before that, the system moves from  $s_i$  to some state  $s_j$ , emits  $O_{t+1}$  there, and explains the rest of the sequence from  $s_j$ ; the derivation is the forward one run in reverse.

## The most probable state at each time can be inconsistent

The forward and backward variables together give the probability of each state given all the observations,

$$\begin{aligned}\gamma_t(i) &= P(q_t = s_i \mid O, \lambda) = \frac{P(O \mid q_t = s_i, \lambda) P(q_t = s_i \mid \lambda)}{P(O \mid \lambda)} \\ &= \frac{\alpha_t(i) \beta_t(i)}{\sum_{j=1}^N \alpha_t(j) \beta_t(j)}.\end{aligned}$$

## The most probable state at each time can be inconsistent

The forward variable  $\alpha_t(i)$  explains the starting part of the sequence until time  $t$ , ending in  $s_i$ , and the backward variable  $\beta_t(i)$  explains the remaining part of the sequence up to time  $T$ , starting from  $s_i$ . Their product is the joint probability,

$$\begin{aligned} P(O, q_t = s_i | \lambda) &= P(O_{1:t}, q_t = s_i | \lambda) P(O_{t+1:T} | q_t = s_i, O_{1:t}, \lambda) \\ &= \alpha_t(i) \beta_t(i), \end{aligned}$$

because given the state at time  $t$  the later observations are independent of the earlier ones. The denominator  $\sum_j \alpha_t(j) \beta_t(j)$  equals  $P(O | \lambda)$  for every  $t$ .

## The most probable state at each time can be inconsistent

We could try to estimate the state by choosing, for each  $t$  separately,

$$\text{greedy: } q_t^* = \arg \max_i \gamma_t(i).$$

This ignores the relations between neighbouring states, and the result may be inconsistent: it can choose  $q_t^* = s_i$  and  $q_{t+1}^* = s_j$  although  $a_{ij} = 0$ , so that the chosen sequence has probability zero.

The example of the figure is an urn HMM in which the urns are visited in the cycle  $1, 2, 3, 1, \dots$ . From each urn the system stays with probability 0.6 or moves to the next urn with probability 0.4, and it never moves backwards, so  $a_{13} = a_{32} = a_{21} = 0$ . Each urn gives its own colour with probability 0.8 and each other colour with probability 0.1, and  $\pi$  is uniform. The observations are red, green, green, blue.

## The most probable state at each time can be inconsistent

The probabilities  $\gamma_t(i)$  of the three states are

|         | $s_1$       | $s_2$       | $s_3$       |
|---------|-------------|-------------|-------------|
| $t = 1$ | <b>0.42</b> | 0.25        | 0.33        |
| $t = 2$ | 0.23        | 0.25        | <b>0.52</b> |
| $t = 3$ | 0.25        | 0.23        | <b>0.52</b> |
| $t = 4$ | 0.25        | <b>0.42</b> | 0.33        |

and the most probable state at each time, in bold olive, gives the sequence  $s_1, s_3, s_3, s_2$ . The red ball at  $t = 1$  points to urn 1, and the green balls at  $t = 2, 3$  point to urn 3; separately, each choice is the best. Together they need the transitions  $a_{13}$  and  $a_{32}$ , both zero, and the sequence has probability zero. Choosing each state separately ignores its neighbours.



## The Viterbi algorithm finds the best state sequence by DP

The Viterbi algorithm (Viterbi, 1967) exploits the linear structure again, and finds the whole state sequence at once. Define  $\delta_t(i)$  as the probability of the highest-probability path that accounts for the first  $t$  observations and ends in  $s_i$ ,

$$\delta_t(i) = \max_{q_1, \dots, q_{t-1}} P(q_1, \dots, q_{t-1}, q_t = s_i, O_1, \dots, O_t | \lambda).$$

It is the forward variable with the sum over the earlier states replaced by a maximum. The best state sequence maximises  $P(Q | O, \lambda) = P(Q, O | \lambda) / P(O | \lambda)$ , and since  $P(O | \lambda)$  does not depend on  $Q$ , it also maximises the joint probability  $P(Q, O | \lambda)$ , which is what  $\delta$  tracks.

# The Viterbi algorithm finds the best state sequence by DP

It is calculated recursively, in four steps:

1. initialise:  $\delta_1(i) = \pi_i b_i(O_1)$ ,  $\psi_1(i) = 0$ ;
2. recursion, for  $t = 2, \dots, T$ :  $\delta_t(j) = \max_i \delta_{t-1}(i) a_{ij} b_j(O_t)$ ,  
 $\psi_t(j) = \arg \max_i \delta_{t-1}(i) a_{ij}$ ;
3. terminate:  $p^* = \max_i \delta_T(i)$ ,  $q_T^* = \arg \max_i \delta_T(i)$ ;
4. backtrack:  $q_t^* = \psi_{t+1}(q_{t+1}^*)$  for  $t = T - 1, T - 2, \dots, 1$ .

The back-pointer  $\psi_t(j)$  keeps track of the state at time  $t - 1$  that maximises  $\delta_t(j)$ , the best previous state; backtracking follows it from the end. The cost is the same as for the forward algorithm, about  $N^2 T$  operations, and  $p^* = P(Q^*, O \mid \lambda)$  is the probability of the best path together with the observations.

## Viterbi is dynamic programming on minus log probabilities

Taking minus the logarithm turns the product into a sum of costs,

$$\begin{aligned} -\log P(Q, O \mid \lambda) = & \left[ -\log \pi_{q_1} - \log b_{q_1}(O_1) \right] \\ & + \sum_{t=2}^T \left[ -\log a_{q_{t-1}q_t} - \log b_{q_t}(O_t) \right]. \end{aligned}$$

Each bracket in the sum couples two neighbouring times, and the first bracket depends on  $q_1$  alone and can be added into the first pair. This is the chain cost  $\varphi$  of dynamic programming, with

$$\varphi_{t-1,t}(q_{t-1}, q_t) = -\log a_{q_{t-1}q_t} - \log b_{q_t}(O_t).$$

The maximum of a product is the minimum of a sum of costs, and the Viterbi recursion is the forward pass with  $h_t = -\log \delta_t$ . The first bracket can be added into the first pair, which starts the pass from  $h_1 = 0$  as before, or, equivalently, the pass can start from  $h_1 = -\log \delta_1$  and use  $\varphi_{t-1,t}$  as written. A zero transition probability is an infinite cost, a road that cannot be taken.

## Viterbi is dynamic programming on minus log probabilities

The letters change between the two settings: the  $N$  variables of the DP chain are the  $T$  times of the HMM, and the  $k$  values of each variable are its  $N$  states. Viterbi is DP on the chain of  $T$  times, with the  $N$  states as the values of each variable.

## The Viterbi algorithm finds the best state sequence by DP

On the urn example the Viterbi path is  $s_3, s_3, s_3, s_3$ : all four balls drawn from urn 3, which is mostly green. The red and the blue balls are then two unlikely draws, with probability 0.1 each, but every transition is allowed, and

$$\begin{aligned} p^* &= \pi_3 b_3(\text{red}) a_{33} b_3(\text{green}) a_{33} b_3(\text{green}) a_{33} b_3(\text{blue}) \\ &= \frac{1}{3} \times 0.1 \times (0.6 \times 0.8)^2 \times 0.6 \times 0.1 \approx 4.6 \times 10^{-4}. \end{aligned}$$

A brute-force search over the  $3^4 = 81$  state sequences gives the same maximum. Two back-pointers are ties, into  $s_2$  at  $t = 3$  and at  $t = 4$ ; they are broken by taking the first index, and neither lies on the Viterbi path.

In the figure the pale teal edges are the back-pointers  $\psi_t(j)$ : for each state at each time, the edge to its best previous state. The circle marks the best final state, and the teal arrows follow  $\psi$  back from it.

The Viterbi algorithm finds the best state sequence by DP

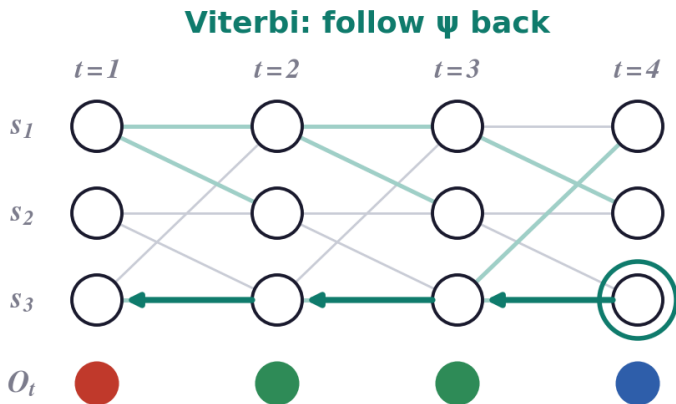


Figure 26: The Viterbi algorithm on the urn example. Pale teal: the edge from each state at each time to its best previous state  $\psi_t(j)$ . The circle marks the best final state, and the teal arrows follow  $\psi$  back from it: the path  $s_3, s_3, s_3, s_3$ .

# The Baum-Welch algorithm learns an HMM by EM

Learning the parameters of an HMM from the observations alone is a problem with hidden variables, the states, and the Baum-Welch algorithm (Baum, Petrie, Soules and Weiss, 1970) solves it by EM. At each iteration,

- ▶ E-step: compute, given the current  $\lambda = (A, B, \pi)$ , the probabilities  $\zeta_t(i, j)$  of pairs of consecutive states and  $\gamma_t(i)$  of single states;
- ▶ M-step: recalculate  $\lambda$  given  $\zeta_t(i, j)$  and  $\gamma_t(i)$ ;

and alternate the two steps until convergence. The quantities are expectations of indicator variables,

$$Z_i^t = \begin{cases} 1 & \text{if } q_t = s_i, \\ 0 & \text{otherwise,} \end{cases} \quad Z_{ij}^t = \begin{cases} 1 & \text{if } q_t = s_i, \ q_{t+1} = s_j, \\ 0 & \text{otherwise.} \end{cases}$$

# The Baum-Welch algorithm learns an HMM by EM

For an observable Markov model the indicators are known and are 0 or 1, and the counts of the observable model are their sums over time. With hidden states they are estimated in the E-step by their expectations under the posterior,

$$\begin{aligned}\text{E-step: } \gamma_t(i) &= E[Z_i^t] = P(q_t = s_i \mid O, \lambda), \\ \zeta_t(i, j) &= E[Z_{ij}^t] = P(q_t = s_i, q_{t+1} = s_j \mid O, \lambda).\end{aligned}$$

The letter  $Z$  here is an indicator, unrelated to the partition function  $Z$  of the Boltzmann machine.

## The E-step computes $\zeta$ from $\alpha$ and $\beta$

The probability of a pair of consecutive states given all the observations follows from the forward and backward variables,

$$\zeta_t(i, j) = \frac{\alpha_t(i) a_{ij} b_j(O_{t+1}) \beta_{t+1}(j)}{P(O | \lambda)} :$$

the forward variable explains the observations up to time  $t$  and ends in  $s_i$ , the factor  $a_{ij} b_j(O_{t+1})$  moves to  $s_j$  and emits  $O_{t+1}$ , and  $\beta_{t+1}(j)$  explains the rest of the sequence from  $s_j$ . Summing over  $j$  gives back the probability of one state,

$$\sum_{j=1}^N \zeta_t(i, j) = \gamma_t(i) \quad \text{for } t \leq T - 1.$$

One forward pass and one backward pass, each of about  $N^2 T$  operations, therefore compute every expectation the M-step needs.

## The Baum-Welch algorithm learns an HMM by EM

The M-step counts the expected number of transitions from  $s_i$  to  $s_j$ ,  $\sum_t \zeta_t(i, j)$ , and the expected number of transitions from  $s_i$ ,  $\sum_t \gamma_t(i)$ , and uses these soft counts in place of the real counts of the observable model:

$$\begin{aligned}\text{M-step: } \hat{\pi}_i &= \gamma_1(i), & \hat{a}_{ij} &= \frac{\sum_{t=1}^{T-1} \zeta_t(i, j)}{\sum_{t=1}^{T-1} \gamma_t(i)}, \\ \hat{b}_j(m) &= \frac{\sum_{t=1}^T \gamma_t(j) I(O_t = v_m)}{\sum_{t=1}^T \gamma_t(j)}.\end{aligned}$$

The estimate  $\hat{b}_j(m)$  is the expected number of times the system is in  $s_j$  and emits  $v_m$ , divided by the expected number of times it is in  $s_j$ .

# The Baum-Welch algorithm learns an HMM by EM

For multiple observation sequences  $X = \{O^k : k = 1, \dots, K\}$ , with  $P(X | \lambda) = \prod_k P(O^k | \lambda)$ , the soft counts are summed over the sequences as well:

$$\hat{\pi}_i = \frac{1}{K} \sum_k \gamma_1^k(i), \quad \hat{a}_{ij} = \frac{\sum_k \sum_{t=1}^{T_k-1} \zeta_t^k(i, j)}{\sum_k \sum_{t=1}^{T_k-1} \gamma_t^k(i)},$$

$$\hat{b}_j(m) = \frac{\sum_k \sum_{t=1}^{T_k} \gamma_t^k(j) I(O_t^k = v_m)}{\sum_k \sum_{t=1}^{T_k} \gamma_t^k(j)},$$

where  $T_k$  is the length of the  $k$ -th sequence.

## The M-step is counting with soft counts

Each M-step maximises the expected log-likelihood of the complete data, the observations together with the states, under the posterior of the E-step. With the indicators it reads

$$\log P(O, Q | \lambda) = \sum_i Z_i^1 \log \pi_i + \sum_{t=1}^{T-1} \sum_{i,j} Z_{ij}^t \log a_{ij} + \sum_{t=1}^T \sum_j Z_j^t \log b_j(O_t).$$

It is linear in the indicators, so its expectation under the posterior replaces each  $Z_i^t$  by  $\gamma_t(i)$  and each  $Z_{ij}^t$  by  $\zeta_t(i, j)$ . The result has the form of the log-likelihood of an observable Markov model, with fractional counts in place of whole ones.

The counting argument then goes through unchanged. For each  $i$ , maximising  $\sum_j (\sum_t \zeta_t(i, j)) \log a_{ij}$  subject to  $\sum_j a_{ij} = 1$  gives  $\hat{a}_{ij}$  proportional to  $\sum_t \zeta_t(i, j)$ , normalised by  $\sum_j \sum_t \zeta_t(i, j) = \sum_{t=1}^{T-1} \gamma_t(i)$ , and the same argument for each  $b_j$  and for  $\pi$  gives the other two estimates.

## The Baum-Welch algorithm learns an HMM by EM

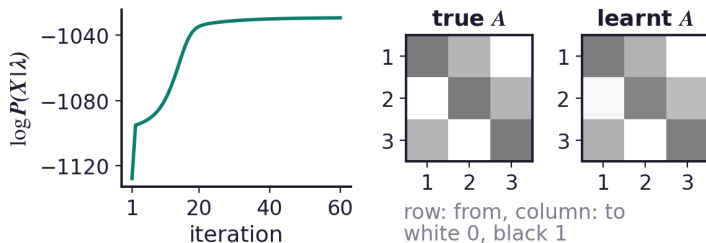
As for any EM with exact E-steps and exact M-steps, the likelihood  $P(X | \lambda)$  never decreases from one iteration to the next (Baum et al., 1970). The likelihood values therefore converge, since they are bounded above by 1. Under regularity conditions the parameters approach a stationary point of the likelihood, a point where its gradient is zero, which is usually a local maximum and may lie on the boundary where some probabilities are 0; which one is reached depends on the starting  $\lambda$ .

In the run of the figure, 40 sequences of 25 draws are generated from the urn HMM of the decoding example, and Baum-Welch starts from random  $A$  and  $B$ . Over 60 iterations  $\log P(X | \lambda)$  rises from  $-1128.1$  to  $-1029.2$  and never falls. EM can find the states in any order, so the learnt states are matched to the urns by their most likely colour; the learnt transition matrix is then

$$\hat{A} = \begin{pmatrix} 0.59 & 0.41 & 0.00 \\ 0.07 & 0.56 & 0.37 \\ 0.42 & 0.00 & 0.58 \end{pmatrix}, \quad \text{against} \quad A = \begin{pmatrix} 0.6 & 0.4 & 0 \\ 0 & 0.6 & 0.4 \\ 0.4 & 0 & 0.6 \end{pmatrix}.$$

# The Baum-Welch algorithm learns an HMM by EM

**the log-likelihood never decreases**



**Figure 27:** Baum-Welch on 40 sequences of 25 draws from the urn HMM, starting from random parameters. The plot is the log-likelihood  $\log P(X|\lambda)$  at each iteration. The two grids are the true transition matrix  $A$  and the learnt one: rows are the state moved from, columns the state moved to, and the shading runs from white for 0 to black for 1.

## HMMs detect highlights in baseball game video

HMMs have been applied to computer vision for problems involving time sequences, such as modelling tennis strokes (Yamato, Ohya and Ishii, 1992) and sequences of actions in a baseball game. Chang, Han and Gong (2002) detect highlights, such as home runs, in broadcast baseball video.

The video is first segmented into scene shots. A scene shot is a stretch of video taken by one camera with continuous camera motion. Broadcast cameras are mounted at fixed places in the stadium, each covering part of the field, so the shots fall into a few types, which share their statistics from game to game. The paper uses seven types of scene shot: pitch view, catch overview, catch close-up, running overview, running close-up, audience view and touch-base close-up.

Most highlights are composed of certain types of scene shot, in a typical order. A typical home run, for example, is a pitch view, followed by an audience view, and then a running close-up.

## HMMs detect highlights in baseball game video

The observations are cheap features, computed from every frame of a shot, chosen so that they can be computed efficiently and reliably:

- ▶ a field descriptor: the frame is divided into  $3 \times 3$  blocks, and each block is marked as field if grass or sand is detected in it;
- ▶ an edge descriptor: the blocks with many edge pixels, which mark highly textured regions such as the audience or the players;
- ▶ the amounts of grass and sand, detected by colour;
- ▶ the camera motion: the pan, tilt and zoom (the horizontal and vertical turning of the camera, and the change of its focal length), estimated from consecutive frames;
- ▶ the player height, the height of boxes of vertical edges that are detected consistently through the shot.

Harder tasks, such as identifying the players or the stadium, are avoided.

## HMMs detect highlights in baseball game video

A classifier gives the probability that a shot is of view type  $V_i$ ,  $i = 1, \dots, 7$ , given its measurements  $M_1, \dots, M_n$ . By Bayes' rule, and assuming the measurements are independent given the view type,

$$p(V_i | M_1, \dots, M_n) = \frac{p(V_i) \prod_{k=1}^n p(M_k | V_i)}{\sum_{i'} p(V_{i'}) \prod_{k=1}^n p(M_k | V_{i'})}.$$

The prior  $p(V_i)$  of each view type is estimated from the training data, and each  $p(M_k | V_i)$  is represented by a histogram learnt from training shots of that type. For a whole shot the paper builds the histogram of each feature over the shot's frames and compares it with the learnt histogram by histogram intersection, the sum over bins of the smaller of the two values, and defines  $p(M_k | V_i)$  from this similarity. In their experiments this worked better than treating the frames as independent and multiplying their probabilities.

## HMMs detect highlights in baseball game video

In HMM terms the view type of shot  $t$  is the hidden state  $q_t$ , the features of shot  $t$  are the observation  $O_t$ , and the observation probability  $b_j(O_t)$  is the histogram model of view type  $j$ . The paper writes  $V$  and  $M$ , unrelated to the alphabet  $V$  and the number of symbols  $M$  of the HMM; these notes use the HMM letters  $q$  and  $O$ .

## HMMs detect highlights in baseball game video

The view classifier alone is reliable for some types and poor for others. The precision of a view type, a measure of detection unrelated to the precision of a Gaussian, is the fraction of the shots classified as that type that really are of that type, and its recall is the fraction of the shots of that type that are classified as it. Pitch views were classified with precision 0.90 and recall 0.89, touch-base close-ups with precision 0.44 and recall 0.53: those shots vary a lot from one to another, and the features are less discriminative for them.

The HMM adds the context of neighbouring shots, and with it, highlight detection can be easier than classifying every view perfectly: under the home-run model, a pitch view followed by an audience view makes a running close-up the likely next shot, even when its own features are ambiguous.

# HMMs detect highlights in baseball game video



pitch view



catch overview



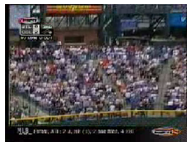
catch close-up



running overview



running close-up



audience view



touch-base close-up

**Figure 28:** The seven types of scene shot of Chang, Han and Gong (2002), one example frame each, from their Figure 1.

## One HMM per type of highlight: the most probable wins

The paper considers four types of highlight: home run, nice hit, nice catch, and play within the diamond, in which the ball stays inside the diamond, the square of the infield marked by the four bases. For each type  $c$  it builds an HMM  $\lambda_c$  whose states are the view types. Learning it only requires the transition probabilities and the initial distribution, estimated from training highlights of that type, since the observation model, the view classifier, is shared by the four models.

# One HMM per type of highlight: the most probable wins

Detection proceeds in four steps:

1. for each scene shot, compute the observations;
2. compute the view classification probabilities, for each shot and each view type;
3. for each highlight model  $\lambda_c$ , compute the probability of the optimal view sequence with the Viterbi algorithm; the paper describes it as the probability of the best view sequence given the observations, and in the notation of these notes it is the Viterbi score

$$p_c^* = \max_Q P(Q, O \mid \lambda_c);$$

4. if the best score passes a threshold, a highlight is detected, and its type is  $c^* = \arg \max_c p_c^*$ .

## One HMM per type of highlight: the most probable wins

Comparing the models is model selection. With two alternative models of the data, select model 1 if  $P(O | \lambda_1) > P(O | \lambda_2)$  and model 2 otherwise: detect which model generated the sequence. The paper uses the Viterbi score of the best path in the place of  $P(O | \lambda)$ , the sum over all paths; both are computed by DP on the same chain. Isolated-word speech recognition works the same way, with one HMM per word, trained on recordings of that word, and each spoken word assigned to the model that gives it the highest probability.

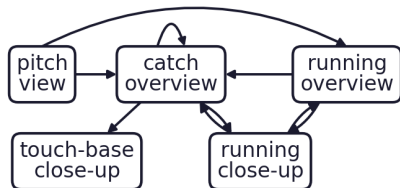
## One HMM per type of highlight: the most probable wins

The system was tested on six baseball games, about 18 hours of video recorded from television, half used for training and half for testing, and the paper reports detecting 5 of the 6 home runs. The precision and recall of detection were 0.71 and 0.83 for home runs, 0.68 and 0.75 for catches, 0.66 and 0.83 for hits, and 0.40 and 0.67 for plays within the diamond. The authors point out the limit of low-level features: a fine distinction, such as a close grounder (a ball hit along the ground) against a nice run onto the base, needs higher-level features, such as detected players, or other sources of information, such as the audio and the captions.

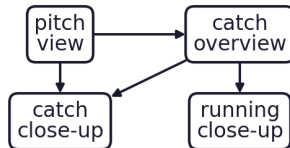
The figure shows the four highlight models. Each box is a view type, a state of the HMM, and each arrow an allowed transition.

## One HMM per type of highlight: the most probable wins

**nice hit**



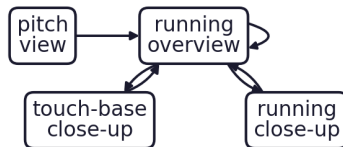
**nice catch**



**home run**



**play within the diamond**



**Figure 29:** The four highlight HMMs of Chang, Han and Gong (2002), Figure 2, redrawn with the same nodes and arrows, since the text of the original cannot be read at this size. Each box is a view type, a state of the HMM, and each arrow an allowed transition; a loop is a transition from a state to itself.

# The HMM and the Bayes-Kalman filter share one graph

The two models of this lecture have the same graph: the hidden states form a Markov chain, and each state emits one observation. They differ in the kind of state.

- ▶ In the Bayes-Kalman filter the state is continuous. With Gaussian models the recursion updates a mean and a variance, two numbers, at every step.
- ▶ In the HMM there are  $N$  discrete states. The forward recursion updates  $N$  numbers, one  $\alpha$  for each state, and the motion and observation models are tables.

Both graphs are chains, so inference in both is dynamic programming. The forward algorithm is the Bayes-Kalman filter for a discrete state, and the Kalman update is the forward algorithm for a Gaussian one.

Markov models and HMMs are exponential distributions whose sufficient statistics are counts, of transitions and of emissions; they are exponential models without and with hidden variables. Their graphs have no closed loops, so dynamic programming can be used for inference and, inside EM, for learning.

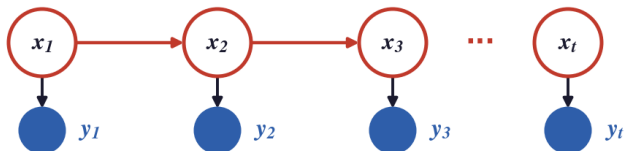
## The HMM and the Bayes-Kalman filter share one graph

The Markov model of a sequence is also the oldest model of language: Markov models of words were early language models. An  $n$ -gram model is a Markov model of order  $n - 1$ : it predicts the next word from the previous  $n - 1$  words, with probabilities estimated by counting, as for the observable Markov model.

Modern language models also predict the next word, from a much longer context. Recurrent networks such as long short-term memory (LSTM) networks carry a learnt continuous state from one word to the next, in the manner of the hidden state of a filter, and the large language models of today use attention to look at a context of thousands of words at once.

# The HMM and the Bayes-Kalman filter share one graph

## Bayes-Kalman: a continuous state



## HMM: one of N discrete states

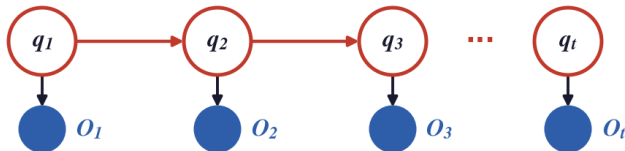


Figure 30: The Bayes-Kalman filter and the HMM have the same graph. The hidden states (red) form a chain, and each emits one observation (blue): a continuous state  $x_t$  with its observation  $y_t$ , and a discrete state  $q_t$  with an observation symbol  $o_t$ .

# Summary

- ▶ The Bayes-Kalman filter tracks a changing state: predict with the motion model, correct with Bayes' rule.
- ▶ With Gaussians it keeps a mean and a variance; the update is a weighted average, as in cue combination.
- ▶ Physical world models use the same graph: physics moves the world state, graphics renders it.
- ▶ Visual de-animation inverts graphics with a network and predicts with a physics engine.

# Summary

- ▶ Dynamic programming minimises a sum of neighbour terms: a forward pass, then a backward pass.
- ▶ It costs about  $k^2N$  operations, and applies to any graph without closed loops.
- ▶ An HMM has hidden states with Markov transitions, and one observation from each state.
- ▶ The forward algorithm evaluates and Viterbi decodes, both by DP; Baum-Welch learns by EM.

# References

- [1] S. Anstis and V. S. Ramachandran. Visual inertia in apparent motion. *Vision Research*, 27(5):755–764, 1987.
- [2] S. N. J. Watamaniuk, S. P. McKee and N. M. Grzywacz. Detecting a trajectory embedded in random-direction motion noise. *Vision Research*, 35(1):65–77, 1995.
- [3] A. L. Yuille, P.-Y. Burgi and N. M. Grzywacz. Visual motion estimation and prediction: a probabilistic network model for temporal coherence. *International Conference on Computer Vision*, 973–978, 1998.
- [4] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- [5] Y. C. Ho and R. C. K. Lee. A Bayesian approach to problems in stochastic estimation and control. *IEEE Transactions on Automatic Control*, 9(4):333–339, 1964.

# References

- [6] A. L. Yuille and D. Kersten. *Probabilities on Graphs: Directed Graphs II*. Lecture notes, Probabilistic Models of the Visual Cortex, Johns Hopkins University, 2025.
- [7] E. D. Dickmanns. The development of machine vision for road vehicles in the last decade. *IEEE Intelligent Vehicle Symposium*, 268–281, 2002.
- [8] T. Lu, T. Shu, A. Yuille, D. Khashabi and J. Chen. Generative world explorer. *International Conference on Learning Representations*, 2025.
- [9] J. Wu. *Physical Scene Understanding*. Lecture slides.
- [10] J. Wu, E. Lu, P. Kohli, W. T. Freeman and J. B. Tenenbaum. Learning to see physics via visual de-animation. *Advances in Neural Information Processing Systems*, 153–164, 2017.

# References

- [11] R. Bellman. *Dynamic Programming*. Princeton University Press, 1957.
- [12] Y. Ohta and T. Kanade. Stereo by intra- and inter-scanline search using dynamic programming. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 7(2):139–154, 1985.
- [13] D. Geiger, B. Ladendorf and A. Yuille. Occlusions and binocular stereo. *International Journal of Computer Vision*, 14(3):211–226, 1995.
- [14] A. L. Yuille. *Vision as Bayesian Inference*: lecture slides on probability distributions on graphs and on hidden Markov models. Johns Hopkins University, 2025.
- [15] S. L. Lauritzen and D. J. Spiegelhalter. Local computations with probabilities on graphical structures and their application to expert systems. *Journal of the Royal Statistical Society B*, 50(2):157–224, 1988.

# References

- [16] L. R. Rabiner. A tutorial on hidden Markov models and selected applications in speech recognition. *Proceedings of the IEEE*, 77(2):257–286, 1989.
- [17] A. J. Viterbi. Error bounds for convolutional codes and an asymptotically optimum decoding algorithm. *IEEE Transactions on Information Theory*, 13(2):260–269, 1967.
- [18] L. E. Baum, T. Petrie, G. Soules and N. Weiss. A maximization technique occurring in the statistical analysis of probabilistic functions of Markov chains. *Annals of Mathematical Statistics*, 41(1):164–171, 1970.
- [19] J. Yamato, J. Ohya and K. Ishii. Recognizing human action in time-sequential images using hidden Markov model. *IEEE Conference on Computer Vision and Pattern Recognition*, 379–385, 1992.
- [20] P. Chang, M. Han and Y. Gong. Extract highlights from baseball game video with hidden Markov models. *IEEE International Conference on Image Processing*, vol. 1, 609–612, 2002.